

第37章 SDIO—SD 卡读写测试

本章参考资料：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》以及 SD 简易规格文件《Physical Layer Simplified Specification V2.0》(版本号：2.00)。

特别说明，本书内容是以 STM32F42x 系列控制器资源讲解。

阅读本章内容之前，建议先阅读 SD 简易规格文件。

37.1 SDIO 简介

SD 卡(Secure Digital Memory Card)在我们生活中已经非常普遍了，控制器对 SD 卡进行读写通信操作一般有两种通信接口可选，一种是 SPI 接口，另外一种就是 SDIO 接口。SDIO 全称是安全数字输入/输出接口，多媒体卡(MMC)、SD 卡、SD I/O 卡都有 SDIO 接口。STM32F42x 系列控制器有一个 SDIO 主机接口，它可以与 MMC 卡、SD 卡、SD I/O 卡以及 CE-ATA 设备进行数据传输。MMC 卡可以说是 SD 卡的前身，现阶段已经用得很少。SD I/O 卡本身不是用于存储的卡，它是指利用 SDIO 传输协议的一种外设。比如 Wi-Fi Card，它主要是提供 Wi-Fi 功能，有些 Wi-Fi 模块是使用串口或者 SPI 接口进行通信的，但 Wi-Fi SDIO Card 是使用 SDIO 接口进行通信的。并且一般设计 SD I/O 卡是可以插入到 SD 的插槽。CE-ATA 是专为轻薄笔记本硬盘设计的硬盘高速通讯接口。

多媒体卡协会网站 www.mmca.org 中提供了有 MMCA 技术委员会发布的多媒体卡系统规范。

SD 卡协会网站 www.sdcard.org 中提供了 SD 存储卡和 SDIO 卡系统规范。

CE-ATA 工作组网站 www.ce-ata.org 中提供了 CE_ATA 系统规范。

随之科技发展，SD 卡容量需求越来越大，SD 卡发展到现在也是有几个版本的，关于 SDIO 接口的设备整体概括见图 37-1。

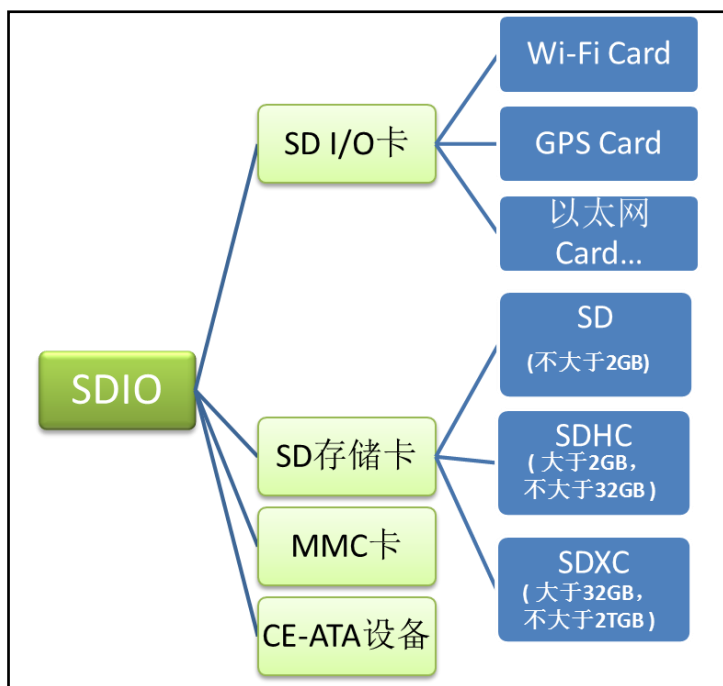


图 37-1 SDIO 接口的设备

关于 SD 卡和 SD I/O 部分内容可以在 SD 协会网站获取到详细的介绍，比如各种 SD 卡尺寸规则、读写速度标示方法、应用扩展等等信息。

本章内容针对 SD 卡使用讲解，对于其他类型卡的应用可以参考相关系统规范实现，所以对于控制器中针对其他类型卡的内容可能在本章中简单提及或者被忽略，本章内容不区分 SDIO 和 SD 卡这两个概念。即使目前 SD 协议提供的 SD 卡规范版本最新是 4.01 版本，但 STM32F42x 系列控制器只支持 SD 卡规范版本 2.0，即只支持标准容量 SD 和高容量 SDHC 标准卡，不支持超大容量 SDXC 标准卡，所以可以支持的最高卡容量是 32GB。

37.2 SD 卡物理结构

一张 SD 卡包括有存储单元、存储单元接口、电源检测、卡及接口控制器和接口驱动器 5 个部分，见图 37-2。存储单元是存储数据部件，存储单元通过存储单元接口与卡控制单元进行数据传输；电源检测单元保证 SD 卡工作在合适的电压下，如出现掉电或上状态时，它会使控制单元和存储单元接口复位；卡及接口控制单元控制 SD 卡的运行状态，它包括有 8 个寄存器；接口驱动器控制 SD 卡引脚的输入输出。

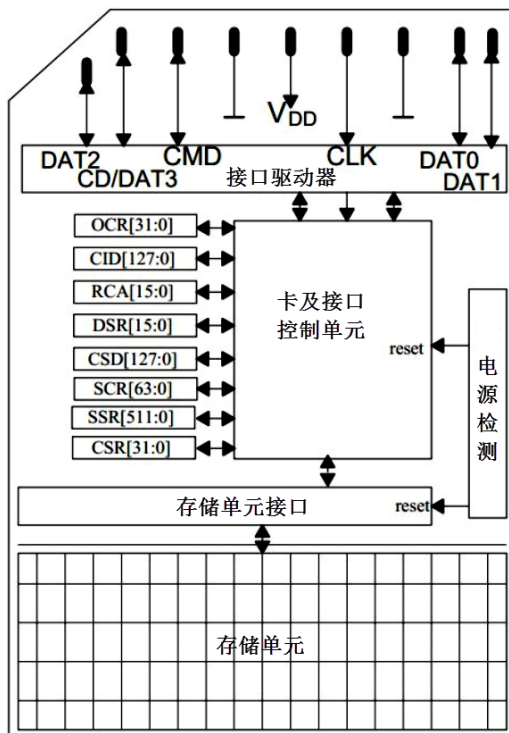


图 37-2 SD 卡物理结构

SD 卡总共有 8 个寄存器，用于设定或表示 SD 卡信息，参考表 37-1。这些寄存器只能通过对应的命令访问，对 SD 卡进行控制操作并不是像操作控制器 GPIO 相关寄存器那样一次读写一个寄存器的，它是通过命令来控制，SDIO 定义了 64 个命令，每个命令都有特殊意义，可以实现某一特定功能，SD 卡接收到命令后，根据命令要求对 SD 卡内部寄存器进行修改，程序控制中只需要发送组合命令就可以实现 SD 卡的控制以及读写操作。

表 37-1 SD 卡寄存器

名称	bit 宽度	描述
CID	128	卡识别号(Card identification number):用来识别的卡的个体号码(唯一的)
RCA	16	相对地址(Relative card address):卡的本地系统地址，初始化时，动态地由卡建议，主机核准。
DSR	16	驱动级寄存器(Driver Stage Register):配置卡的输出驱动
CSD	128	卡的特定数据(Card Specific Data):卡的操作条件信息
SCR	64	SD 配置寄存器(SD Configuration Register):SD 卡特殊特性信息
OCR	32	操作条件寄存器(Operation conditions register)
SSR	512	SD 状态(SD Status):SD 卡专有特征的信息
CSR	32	卡状态(Card Status):卡状态信息

每个寄存器位的含义可以参考 SD 简易规格文件《Physical Layer Simplified Specification V2.0》第 5 章内容。

37.3 SDIO 总线

37.3.1 总线拓扑

SD 卡一般都支持 SDIO 和 SPI 这两种接口，本章内容只介绍 SDIO 接口操作方式，如果需要使用 SPI 操作方式可以参考 SPI 相关章节。另外，STM32F4xx 系列控制器的 SDIO 是不支持 SPI 通信模式的，如果需要用到 SPI 通信只能使用 SPI 外设。

SD 卡总线拓扑参考图 37-3。虽然可以共用总线，但不推荐多卡槽共用总线信号，要求一个单独 SD 总线应该连接一个单独的 SD 卡。

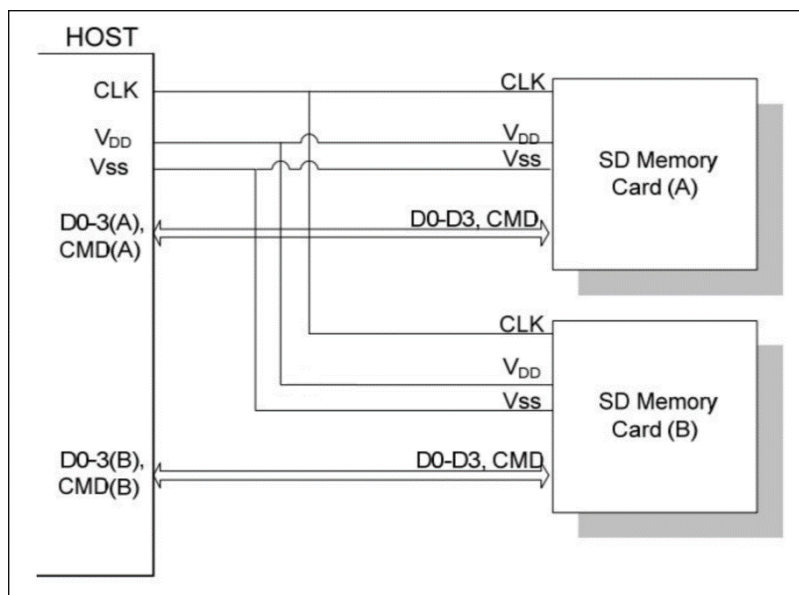


图 37-3 SD 卡总线拓扑

SD 卡使用 9-pin 接口通信，其中 3 根电源线、1 根时钟线、1 根命令线和 4 根数据线，具体说明如下：

- ❑ **CLK**：时钟线，由 SDIO 主机产生，即由 STM32 控制器输出；
- ❑ **CMD**：命令控制线，SDIO 主机通过该线发送命令控制 SD 卡，如果命令要求 SD 卡提供应答(响应)，SD 卡也是通过该线传输应答信息；
- ❑ **D0-3**：数据线，传输读写数据；SD 卡可将 D0 拉低表示忙状态；
- ❑ **VDD、VSS1、VSS2**：电源和地信号。

在之前的 I2C 以及 SPI 章节都有详细讲解了对应的通信时序，实际上，SDIO 的通信时序简单许多，SDIO 不管是主机控制器向 SD 卡传输，还是 SD 卡向主机控制器传输都只以 CLK 时钟线的上升沿为有效。SD 卡操作过程会使用两种不同频率的时钟同步数据，一个是识别卡阶段时钟频率 FOD，最高为 400kHz，另外一个数据传输模式下时钟频率 FPP，默认最高为 25MHz，如果通过相关寄存器配置使 SDIO 工作在高速模式，此时数据传输模式最高频率为 50MHz。

对于 STM32 控制器只有一个 SDIO 主机，所以只能连接一个 SDIO 设备，开发板上集成了一个 Micro SD 卡槽和 SDIO 接口的 WiFi 模块，要求只能使用其中一个设备。SDIO 接

口的 WiFi 模块一般集成有使能线，如果需要用到 SD 卡需要先控制该使能线禁用 WiFi 模块。

37.3.2 总线协议

SD 总线通信是基于命令和数据传输的。通讯由一个起始位(“0”), 由一个停止位(“1”)终止。SD 通信一般是主机发送一个命令(Command), 从设备在接收到命令后作出响应(Response), 如有需要会有数据(Data)传输参与。

SD 总线的基本交互是命令与响应交互, 见图 37-4。

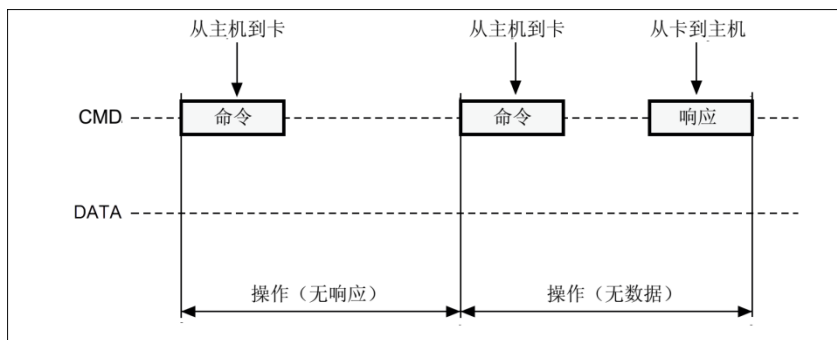


图 37-4 命令与响应交互

SD 数据是以块(Block)形式传输的, SDHC 卡数据块长度一般为 512 字节, 数据可以从主机到卡, 也可以是从卡到主机。数据块需要 CRC 位来保证数据传输成功。CRC 位由 SD 卡系统硬件生成。STM32 控制器可以控制使用单线或 4 线传输, 本开发板设计使用 4 线传输。图 37-5 为主机向 SD 卡写入数据块操作示意。

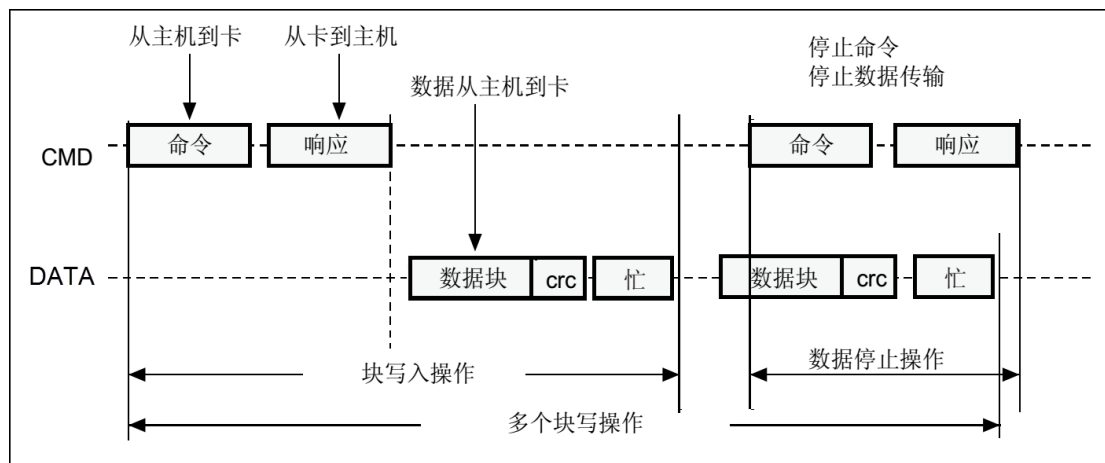


图 37-5 多块写入操作

SD 数据传输支持单块和多块读写, 它们分别对应不同的操作命令, 多块写入还需要使用命令来停止整个写入操作。数据写入前需要检测 SD 卡忙状态, 因为 SD 卡在接收到数据后编程到存储区过程需要一定操作时间。SD 卡忙状态通过把 D0 线拉低表示。

数据块读操作与之类似, 只是无需忙状态检测。

使用 4 数据线传输时, 每次传输 4bit 数据, 每根数据线都必须有起始位、终止位以及 CRC 位, CRC 位每根数据线都要分别检查, 并把检查结果汇总然后在数据传输完后通过 D0 线反馈给主机。

SD 卡数据包有两种格式，一种是常规数据(8bit 宽)，它先发低字节再发高字节，而每个字节则是先发高位再发低位，4 线传输示意如图 37-6。

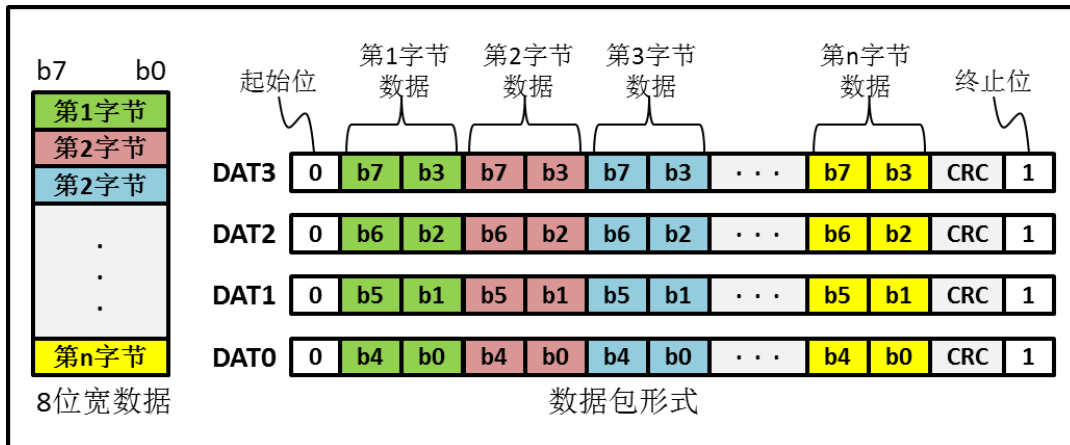


图 37-6 8 位宽数据包传输

4 线同步发送，每根线发送一个字节的其中两个位，数据位在四线顺序排列发送，DAT3 数据线发较高位，DAT0 数据线发较低位。

另外一种数据包发送格式是宽位数据包格式，对 SD 卡而言宽位数据包发送方式是针对 SD 卡 SSR(SD 状态)寄存器内容发送的，SSR 寄存器总共有 512bit，在主机发出 ACMD13 命令后 SD 卡将 SSR 寄存器内容通过 DAT 线发送给主机。宽位数据包格式示意图 37-7。

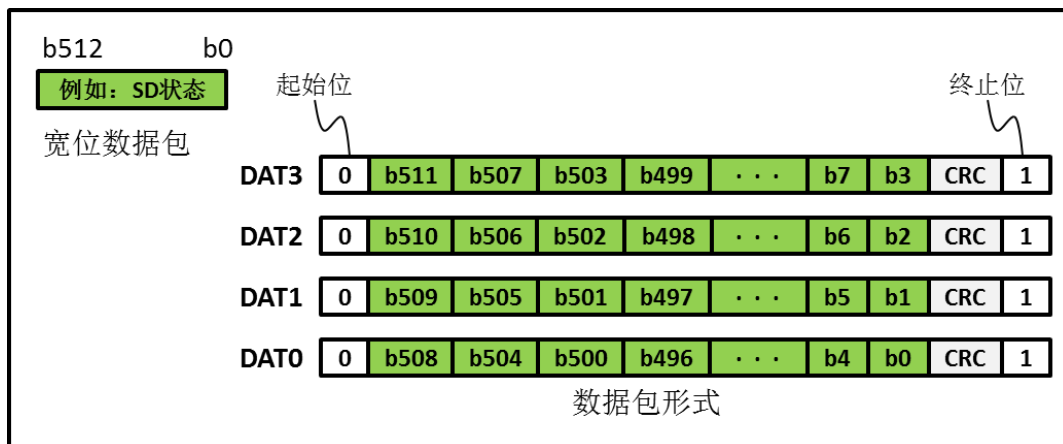


图 37-7 宽位数据包传输

37.3.3 命令

SD 命令由主机发出，以广播命令和寻址命令为例，广播命令是针对与 SD 主机总线连接的所有从设备发送的，寻址命令是指定某个地址设备进行命令传输。

1. 命令格式

SD 命令格式固定为 48bit，都是通过 CMD 线连续传输的（数据线不参与），见图 37-8。

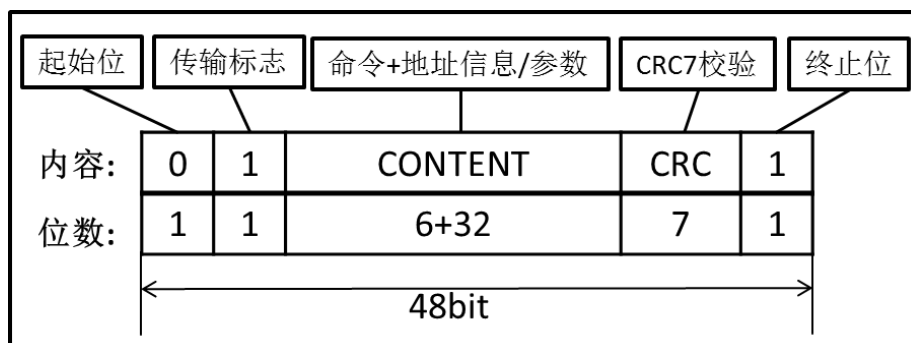


图 37-8 SD 命令格式

SD 命令的组成如下:

- ❑ 起始位和终止位: 命令的主体包含在起始位与终止位之间, 它们都只包含一个数据位, 起始位为 0, 终止位为 1。
- ❑ 传输标志: 用于区分传输方向, 该位为 1 时表示命令, 方向为主机传输到 SD 卡, 该位为 0 时表示响应, 方向为 SD 卡传输到主机。

命令主体内容包括命令、地址信息/参数和 CRC 校验三个部分。

- ❑ 命令号: 它固定占用 6bit, 所以总共有 64 个命令(代号: CMD0~CMD63), 每个命令都有特定的用途, 部分命令不适用于 SD 卡操作, 只是专门用于 MMC 卡或者 SD I/O 卡。
- ❑ 地址/参数: 每个命令有 32bit 地址信息/参数用于命令附加内容, 例如, 广播命令没有地址信息, 这 32bit 用于指定参数, 而寻址命令这 32bit 用于指定目标 SD 卡的地址。
- ❑ CRC7 校验: 长度为 7bit 的校验位用于验证命令传输内容正确性, 如果发生外部干扰导致传输数据个别位状态改变将导致校准失败, 也意味着命令传输失败, SD 卡不执行命令。

2. 命令类型

SD 命令有 4 种类型:

- ❑ 无响应广播命令(bc), 发送到所有卡, 不返回任务响应;
- ❑ 带响应广播命令(bcr), 发送到所有卡, 同时接收来自所有卡响应;
- ❑ 寻址命令(ac), 发送到选定卡, DAT 线无数据传输;
- ❑ 寻址数据传输命令(adtc), 发送到选定卡, DAT 线有数据传输。

另外, SD 卡主机模块系统旨在为各种应用程序类型提供一个标准接口。在此环境中, 需要有特定的客户/应用程序功能。为实现这些功能, 在标准中定义了两种类型的通用命令: 特定应用命令(ACMD)和常规命令(GEN_CMD)。要使用 SD 卡制造商特定的 ACMD 命令如 ACMD6, 需要在发送该命令之前无发送 CMD55 命令, 告知 SD 卡接下来的命令为特定应用命令。CMD55 命令只对紧接的第一个命令有效, SD 卡如果检测到 CMD55 之后的第一条命令为 ACMD 则执行其特定应用功能, 如果检测发现不是 ACMD 命令, 则执行标准命令。

3. 命令描述

SD 卡系统的命令被分为多个类，每个类支持一种“卡的功能设置”。表 37-2 列举了 SD 卡部分命令信息，更多详细信息可以参考 SD 简易规格文件说明，表中填充位和保留位都必须被设置为 0。

虽然没有必须完全记住每个命令详细信息，但越熟悉命令对后面编程理解非常有帮助。

表 37-2 SD 部分命令描述

命令序号	类型	参数	响应	缩写	描述
基本命令 (Class 0)					
CMD0	bc	[31:0]填充位	-	GO_IDLE_STATE	复位所有的卡到 idle 状态。
CMD2	bcr	[31:0]填充位	R2	ALL_SEND_CID	通知所有卡通过 CMD 线返回 CID 值。
CMD3	bcr	[31:0]填充位	R6	SEND_RELATIVE_ADDR	通知所有卡发布新 RCA。
CMD4	bc	[31:16]DSR[15:0]填充位	-	SET_DSR	编程所有卡的 DSR。
CMD7	ac	[31:16]RCA[15:0]填充位	R1b	SELECT/DESELECT_CARD	选择/取消选择 RCA 地址卡。
CMD8	bcr	[31:12]保留位 [11:8]VHS[7:0]检查模式	R7	SEND_IF_COND	发送 SD 卡接口条件，包含主机支持的电压信息，并询问卡是否支持。
CMD9	ac	[31:16]RCA[15:0]填充位	R2	SEND_CSD	选定卡通过 CMD 线发送 CSD 内容
CMD10	ac	[31:16]RCA[15:0]填充位	R2	SEND_CID	选定卡通过 CMD 线发送 CID 内容
CMD12	ac	[31:0]填充位	R1b	STOP_TRANSMISSION	强制卡停止传输
CMD13	ac	[31:16]RCA[15:0]填充位	R1	SEND_STATUS	选定卡通过 CMD 线发送它状态寄存器
CMD15	ac	[31:16]RCA[15:0]填充位	-	GO_INACTIVE_STATE	使选定卡进入“inactive”状态
面向块的读操作 (Class 2)					
CMD16	ac	[31:0]块长度	R1	SET_BLOCK_LEN	对于标准 SD 卡，设置块命令的长度，对于 SDHC 卡块命令长度固定为 512 字节。
CMD17	adtc	[31:0]数据地址	R1	READ_SINGLE_BLOCK	对于标准卡，读取 SEL_BLOCK_LEN 长度字节的块；对于 SDHC 卡，读取 512 字节的块。
CMD18	adtc	[31:0]数据地址	R1	READ_MULTIPLE_BLOCK	连续从 SD 卡读取数据块，直到被 CMD12 中断。块长度同 CMD17。
面向块的写操作 (Class 4)					
CMD24	adtc	[31:0]数据地址	R1	WRITE_BLOCK	对于标准卡，写入 SEL_BLOCK_LEN 长度字节的块；对于 SDHC 卡，写入 512 字节的块。
CMD25	adtc	[31:0]数据地址	R1	WRITE_MULTIPLE_BLOCK	连续向 SD 卡写入数据块，直到被 CMD12 中断。每块长度同 CMD17。
CMD27	adtc	[31:0]填充位	R1	PROGRAM_CSD	对 CSD 的可编程位进行编程
擦除命令 (Class 5)					
CMD32	ac	[31:0]数据地址	R1	ERASE_WR_BLK_START	设置擦除的起始块地址
CMD33	ac	[31:0]数据地址	R1	ERASE_WR_BLK_END	设置擦除的结束块地址
CMD38	ac	[31:0]填充位	R1b	ERASE	擦除预先选定的块
加锁命令 (Class 7)					
CMD42	adtc	[31:0]保留	R1	LOCK_UNLOCK	加锁/解锁 SD 卡
特定应用命令 (Class 8)					
CMD55	ac	[31:16]RCA[15:0]	R1	APP_CMD	指定下个命令为特定应用命令，不

		填充位			是标准命令
CMD56	adtc	[31:1] 填充位 [0] 读/写	R1	GEN_CMD	通用命令, 或者特定应用命令中, 用于传输一个数据块, 最低位为 1 表示读数据, 为 0 表示写数据
SD 卡特定应用命令					
ACMD6	ac	[31:2] 填充位 [1:0] 总线宽度	R1	SET_BUS_WIDTH	定义数据总线宽度 ('00'=1bit, '10'=4bit)。
ACMD13	adtc	[31:0] 填充位	R1	SD_STATUS	发送 SD 状态
ACMD41	Bcr	[32] 保留位 [30] HCS (OCR[30]) [29:24] 保留位 [23:0] VDD 电压 (OCR[23:0])	R3	SD_SEND_OP_COND	主机要求卡发送它的支持信息 (HCS) 和 OCR 寄存器内容。
ACMD51	adtc	[31:0] 填充位	R1	SEND_SCR	读取配置寄存器 SCR

37.3.4 响应

响应由 SD 卡向主机发出, 部分命令要求 SD 卡作出响应, 这些响应多用于反馈 SD 卡的状态。SDIO 总共有 7 个响应类型(代号: R1~R7), 其中 SD 卡没有 R4、R5 类型响应。特定的命令对应有特定的响应类型, 比如当主机发送 CMD3 命令时, 可以得到响应 R6。与命令一样, SD 卡的响应也是通过 CMD 线连续传输的。根据响应内容大小可以分为短响应和长响应。短响应是 48bit 长度, 只有 R2 类型是长响应, 其长度为 136bit。各个类型响应具体情况如表 37-3。

除了 R3 类型之外, 其他响应都使用 CRC7 校验来校验, 对于 R2 类型是使用 CID 和 CSD 寄存器内部 CRC7。

表 37-3 SD 卡响应类型

R1 (正常响应命令)							
描述	起始位	传输位	命令号	卡状态	CRC7	终止位	
Bit	47	46	[45:40]	[39:8]	[7:1]	0	
位宽	1	1	6	32	7	1	
值	"0"	"0"	x	x	x	"1"	
备注	如果有传输到卡的数据，那么在数据线可能有 busy 信号						
R2 (CID, CSD 寄存器)							
描述	起始位	传输位	保留	[127:1]		终止位	
Bit	135	134	[133:128]	127		0	
位宽	1	1	6	x		1	
值	"0"	"0"	"111111"	CID 或者 CSD 寄存器[127:1]位的值		"1"	
备注	CID 寄存器内容作为 CMD2 和 CMD10 响应，CSD 寄存器内容作为 CMD9 响应。						
R3 (OCR 寄存器)							
描述	起始位	传输位	保留	OCR 寄存器	保留	终止位	
Bit	47	46	[45:40]	[39:8]	[7:1]	0	
位宽	1	1	6	32	7	1	
值	"0"	"0"	"111111"	x	"111111"	"1"	
备注	OCR 寄存器的值作为 ACMD41 的响应						
R6 (发布的 RCA 寄存器响应)							
描述	起始位	传输位	CMD3	RCA 寄存器	卡状态位	CRC7	终止位
Bit	47	46	[45:40]	[39:8]		[7:1]	0

位宽	1	1	6	16	16	7	1	
值	"0"	"0"	"000011"	x	x	x	"1"	
备注	专用于命令 CMD3 的响应							
R7(发布的 RCA 寄存器响应)								
描述	起始位	传输位	CMD8	保留	接收电压	检测模式	CRC7	终止位
Bit	47	46	[45:40]	[39:20]	[19:16]	[15:8]	[7:1]	0
位宽	1	1	6	20	4	8	7	1
值	"0"	"0"	"001000"	"00000h"	x	x	x	"1"
备注	专用于命令 CMD8 的响应, 返回卡支持电压范围和检测模式							

37.4 SD 卡的操作模式及切换

37.4.1 SD 卡的操作模式

SD 卡有多个版本, STM32 控制器目前最高支持《Physical Layer Simplified Specification V2.0》定义的 SD 卡, STM32 控制器对 SD 卡进行数据读写之前需要识别卡的种类: V1.0 标准卡、V2.0 标准卡、V2.0 大容量卡或者不被识别卡。

SD 卡系统(包括主机和 SD 卡)定义了两种操作模式: 卡识别模式和数据传输模式。在系统复位后, 主机处于卡识别模式, 寻找总线上可用的 SDIO 设备; 同时, SD 卡也处于卡识别模式, 直到被主机识别到, 即当 SD 卡接收到 SEND_RCA(CMD3)命令后, SD 卡就会进入数据传输模式, 而主机在总线上所有卡被识别后也进入数据传输模式。在每个操作模式下, SD 卡都有几种状态, 参考表 37-4, 通过命令控制实现卡状态的切换。

表 37-4 SD 卡状态与操作模式

操作模式	SD 卡状态
无效模式(Inactive)	无效状态(Inactive State)
卡识别模式(Card identification mode)	空闲状态(Idle State)
	准备状态(Ready State)
	识别状态(Identification State)
数据传输模式(Data transfer mode)	待机状态(Stand-by State)
	传输状态(Transfer State)
	发送数据状态(Sending-data State)
	接收数据状态(Receive-data State)
	编程状态(Programming State)
	断开连接状态(Disconnect State)

37.4.2 卡识别模式

在卡识别模式下, 主机复位所有处于“卡识别模式”的 SD 卡, 确认其工作电压范围, 识别 SD 卡类型, 并且获取 SD 卡的相对地址(卡相对地址较短, 便于寻址)。在卡识别过程中, 要求 SD 卡工作在识别时钟频率 FOD 的状态下。卡识别模式下 SD 卡状态转换如图 37-9。

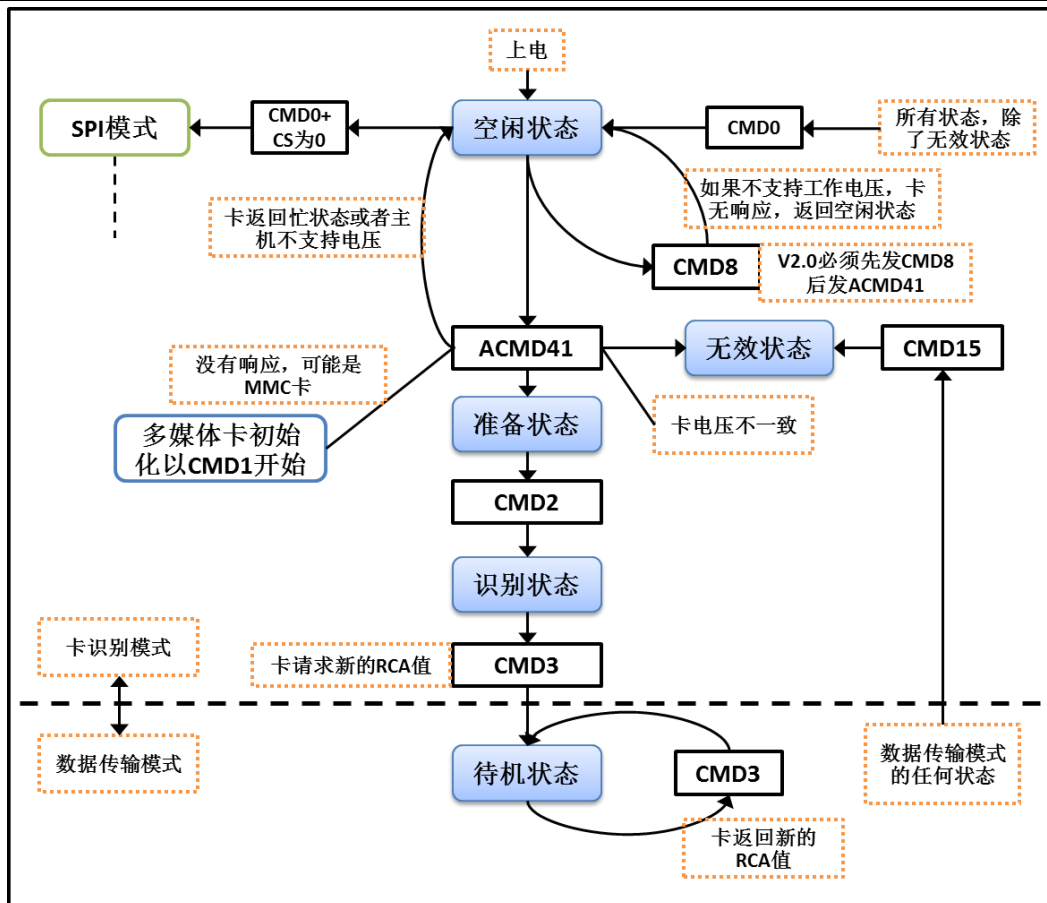


图 37-9 卡识别模式状态转换图

主机上电后，所有卡处于空闲状态，包括当前处于无效状态的卡。主机也可以发送 GO_IDLE_STATE(CMD0)让所有卡软复位从而进入空闲状态，但当前处于无效状态的卡并不会复位。

主机在开始与卡通信前，需要先确定双方在互相支持的电压范围内。SD 卡有一个电压支持范围，主机当前电压必须在该范围可能才能与卡正常通信。SEND_IF_COND(CMD8) 命令就是用于验证卡接口操作条件的(主要是电压支持)。卡会根据命令的参数来检测操作条件匹配性，如果卡支持主机电压就产生响应，否则不响应。而主机则根据响应内容确定卡的电压匹配性。CMD8 是 SD 卡标准 V2.0 版本才有的新命令，所以如果主机有接收到响应，可以判断卡为 V2.0 或更高版本 SD 卡。

SD_SEND_OP_COND(ACMD41)命令可以识别或拒绝不匹配它的电压范围的卡。ACMD41 命令的 VDD 电压参数用于设置主机支持电压范围，卡响应会返回卡支持的电压范围。对于对 CMD8 有响应的卡，把 ACMD41 命令的 HCS 位设置为 1，可以测试卡的容量类型，如果卡响应的 CCS 位为 1 说明为高容量 SD 卡，否则为标准卡。卡在响应 ACMD41 之后进入准备状态，不响应 ACMD41 的卡为不可用卡，进入无效状态。ACMD41 是应用特定命令，发送该命令之前必须先发 CMD55。

ALL_SEND_CID(CMD2)用来控制所有卡返回它们的卡识别号(CID)，处于准备状态的卡在发送 CID 之后就进入识别状态。之后主机就发送 SEND_RELATIVE_ADDR(CMD3)命令，让卡自己推荐一个相对地址(RCA)并响应命令。这个 RCA 是 16bit 地址，而 CID 是

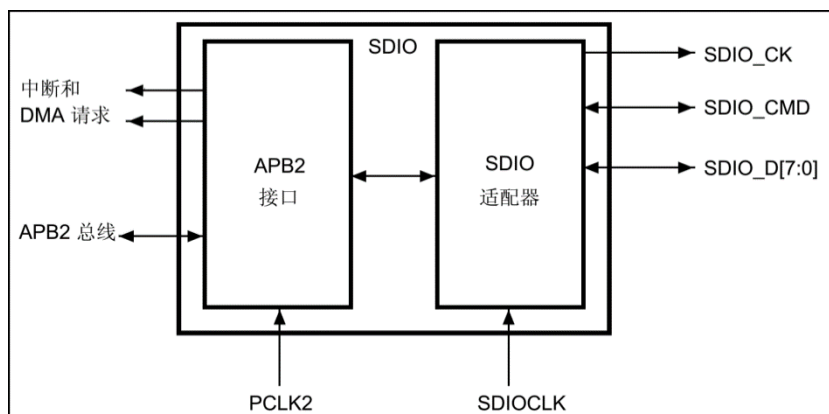


图 37-11 SDIO 功能框图

SDIO 使用两个时钟信号，一个是 SDIO 适配器时钟(SDIOCLK=48MHz)，另外一个 APB2 总线时钟(PCLK2，一般为 84MHz)。

STM32 控制器的 SDIO 是针对 MMC 卡和 SD 卡的主设备，所以预留有 8 根数据线，对于 SD 卡最多用四根数据线。

SDIO 适配器是 SD 卡系统主机部分，是 STM32 控制器与 SD 卡数据通信中间设备。SDIO 适配器由五个单元组成，分别是控制单元、命令路径单元、数据路径单元、寄存器单元以及 FIFO，见图 37-12。

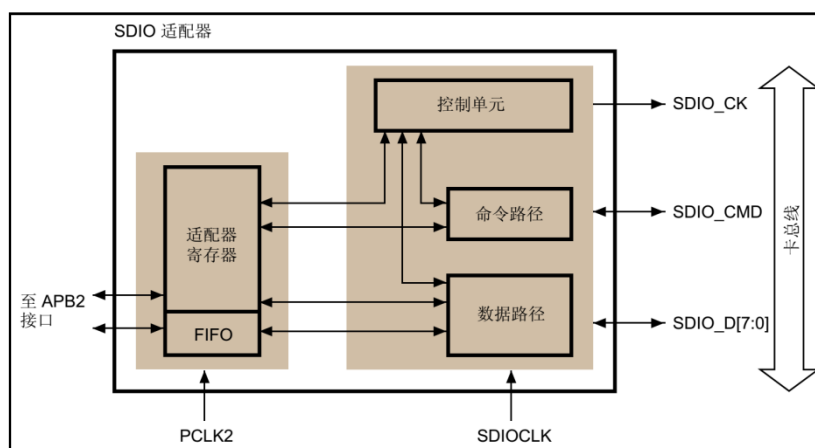


图 37-12 SDIO 适配器框图

1. 控制单元

控制单元包含电源管理和时钟管理功能，结构如图 37-13。电源管理部件会在系统断电和上电阶段禁止 SD 卡总线输出信号。时钟管理部件控制 CLK 线时钟信号生成。一般使用 SDIOCLK 分频得到。

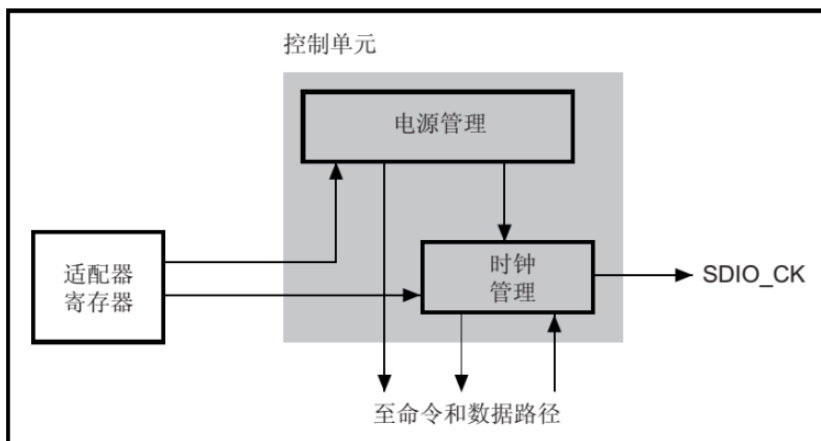


图 37-13 SDIO 适配器控制单元

2. 命令路径

命令路径控制命令发送，并接收卡的响应，结构见图 37-14。

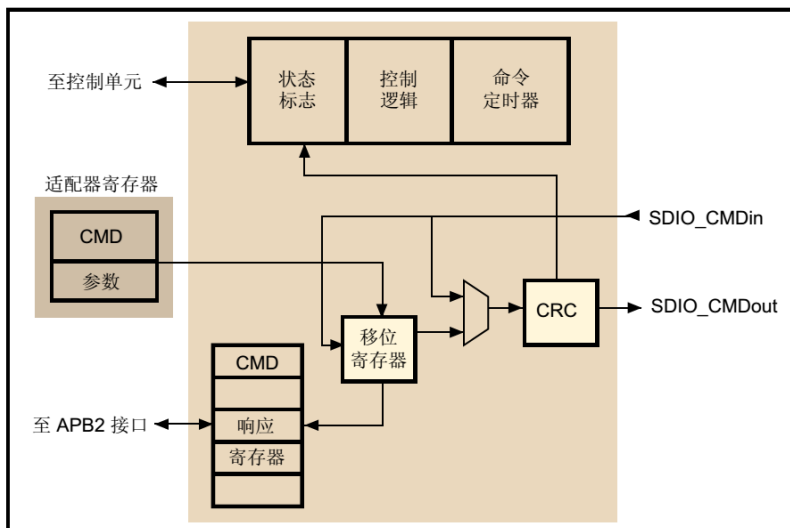


图 37-14 SDIO 适配器命令路径

关于 SDIO 适配器状态转换流程可以参考图 37-9，当 SD 卡处于某一状态时，SDIO 适配器必然处于特定状态与之对应。STM32 控制器以命令路径状态机(CPSM)来描述 SDIO 适配器的状态变化，并加入了等待超时检测功能，以便退出永久等待的情况。CPSM 的描述见图 37-15。

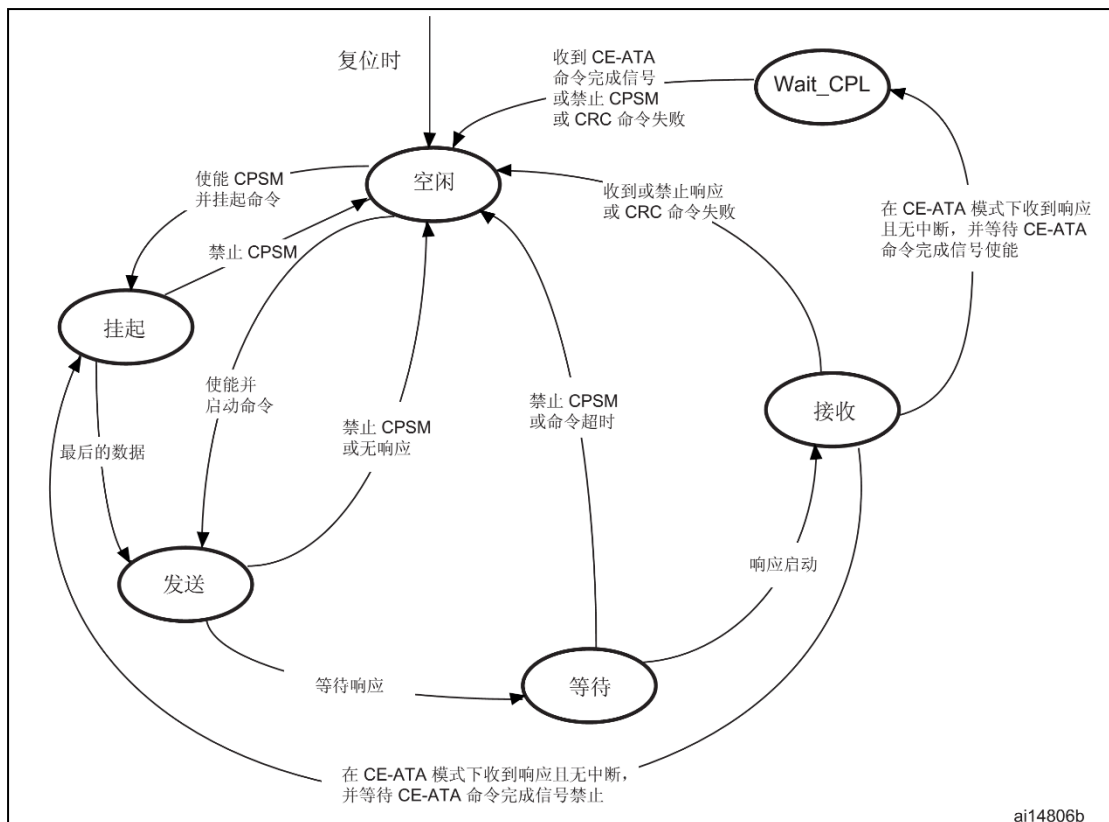


图 37-15 CPSM 状态机描述图

3. 数据路径

数据路径部件负责与 SD 卡相互数据传输，内部结构见图 37-16。

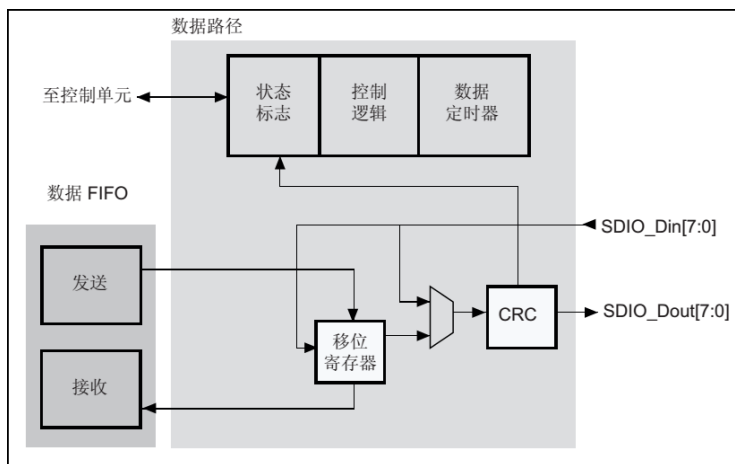


图 37-16 SDIO 适配器数据路径

SD 卡系统数据传输状态转换参考图 37-10，SDIO 适配器以数据路径状态机(DPSM)来描述 SDIO 适配器状态变化情况。并加入了等待超时检测功能，以便退出永久等待情况。发送数据时，DPSM 处于等待发送(Wait_S)状态，如果数据 FIFO 不为空，DPSM 变成发送状态并且数据路径部件启动向卡发送数据。接收数据时，DPSM 处于等待接收状态，当

DPSM 收到起始位时变成接收状态，并且数据路径部件开始从卡接收数据。DPSM 状态机描述见图 37-17。

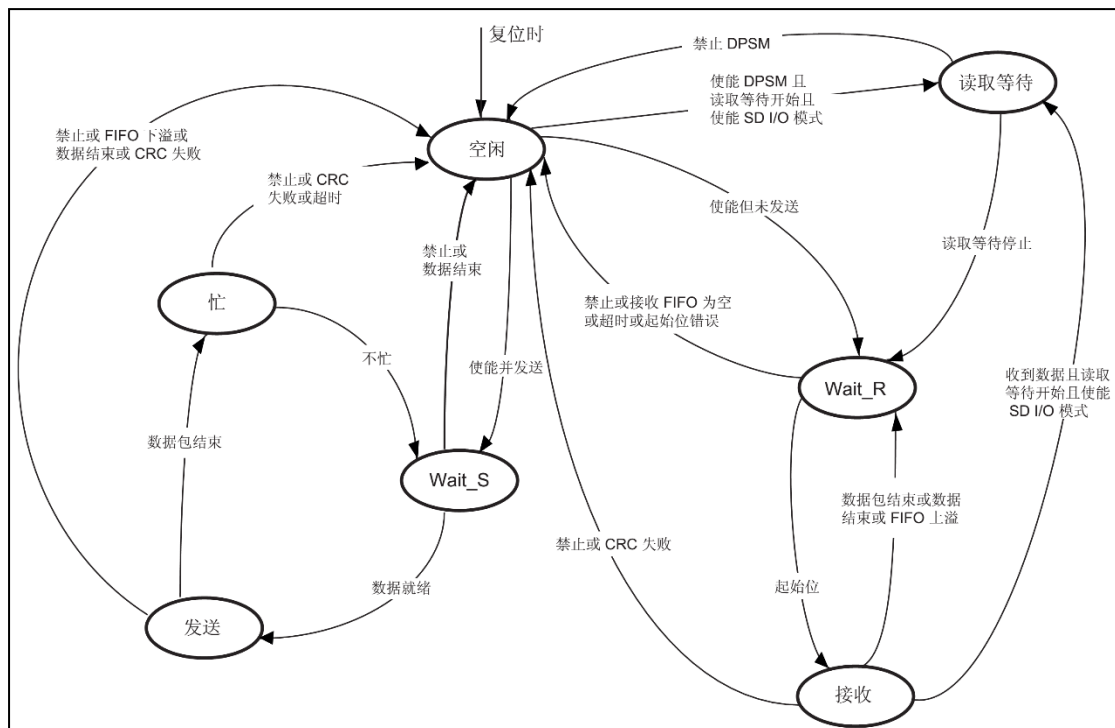


图 37-17 DPSM 状态机描述图

4. 数据 FIFO

数据 FIFO(先进先出)部件是一个数据缓冲器，带发送和接收单元。控制器的 FIFO 包含宽度为 32bit、深度为 32 字的数据缓冲器和发送/接收逻辑。其中 SDIO 状态寄存器 (SDIO_STA) 的 TXACT 位用于指示当前正在发送数据，RXACT 位指示当前正在接收数据，这两个位不可能同时为 1。

❑ 当 TXACT 为 1 时，可以通过 APB2 接口将数据写入到传输 FIFO。

❑ 当 RXACT 为 1 时，接收 FIFO 存放从数据路径部件接收到的数据。

根据 FIFO 空或满状态会把 SDIO_STA 寄存器位值 1，并可以产生中断和 DMA 请求。

5. 适配器寄存器

适配器寄存器包含了控制 SDIO 外设的各种控制寄存器及状态寄存器，内容较多，可以通过 SDIO 提供的各种结构体来了解，这些寄存器的功能都被整合到了结构体或 ST 标准库之中。

37.6 SDIO 初始化结构体

标准库函数对 SDIO 外设建立了三个初始化结构体，分别为 SDIO 初始化结构体 SDIO_InitTypeDef、SDIO 命令初始化结构体 SDIO_CmdInitTypeDef 和 SDIO 数据初始化结构体 SDIO_DataInitTypeDef。这些结构体成员用于设置 SDIO 工作环境参数，并由 SDIO 相

应初始化配置函数或功能函数调用，这些参数将会被写入到 SDIO 相应的寄存器，达到配置 SDIO 工作环境的目的。

初始化结构体和初始化库函数配合使用是标准库精髓所在，理解了初始化结构体每个成员意义基本上就可以对该外设运用自如了。初始化结构体定义在 `stm32f4xx_sdio.h` 文件中，初始化库函数定义在 `stm32f4xx_sdio.c` 文件中，编程时我们可以结合这两个文件内注释使用。

SDIO 初始化结构体用于配置 SDIO 基本工作环境，比如时钟分频、时钟沿、数据宽度等等。它被 `SDIO_Init` 函数使用。

代码清单 37-1 SDIO 初始化结构体

```
1 typedef struct {  
2     uint32_t SDIO_ClockEdge;           // 时钟沿  
3     uint32_t SDIO_ClockBypass;         // 旁路时钟  
4     uint32_t SDIO_ClockPowerSave;      // 节能模式  
5     uint32_t SDIO_BusWide;             // 数据宽度  
6     uint32_t SDIO_HardwareFlowControl; // 硬件流控制  
7     uint8_t  SDIO_ClockDiv;            // 时钟分频  
8 } SDIO_InitTypeDef;
```

各结构体成员的作用介绍如下：

- (1) `SDIO_ClockEdge`：主时钟 `SDIOCLK` 产生 `CLK` 引脚时钟有效沿选择，可选上升沿或下降沿，它设定 `SDIO` 时钟控制寄存器(`SDIO_CLKCR`)的 `NEGEDGE` 位的值，一般选择设置为高电平。
- (2) `SDIO_ClockBypass`：时钟分频旁路使用，可选使能或禁用，它设定 `SDIO_CLKCR` 寄存器的 `BYPASS` 位。如果使能旁路，`SDIOCLK` 直接驱动 `CLK` 线输出时钟；如果禁用，使用 `SDIO_CLKCR` 寄存器的 `CLKDIV` 位值分频 `SDIOCLK`，然后输出到 `CLK` 线。一般选择禁用时钟分频旁路。
- (3) `SDIO_ClockPowerSave`：节能模式选择，可选使能或禁用，它设定 `SDIO_CLKCR` 寄存器的 `PWRSV` 位的值。如果使能节能模式，`CLK` 线只有在总线激活时才有时钟输出；如果禁用节能模式，始终使能 `CLK` 线输出时钟。
- (4) `SDIO_BusWide`：数据线宽度选择，可选 1 位数据总线、4 位数据总线或 8 为数据总线，系统默认使用 1 位数据总线，操作 SD 卡时在数据传输模式下一般选择 4 位数据总线。它设定 `SDIO_CLKCR` 寄存器的 `WIDBUS` 位的值。
- (5) `SDIO_HardwareFlowControl`：硬件流控制选择，可选使能或禁用，它设定 `SDIO_CLKCR` 寄存器的 `HWFC_EN` 位的值。硬件流控制功能可以避免 FIFO 发送上溢和下溢错误。
- (6) `SDIO_ClockDiv`：时钟分频系数，它设定 `SDIO_CLKCR` 寄存器的 `CLKDIV` 位的值，设置 `SDIOCLK` 与 `CLK` 线输出时钟分频系数：

$$\text{CLK 线时钟频率} = \text{SDIOCLK} / ([\text{CLKDIV} + 2])。$$

37.7 SDIO 命令初始化结构体

SDIO 命令初始化结构体用于设置命令相关内容，比如命令号、命令参数、响应类型等等。它被 `SDIO_SendCommand` 函数使用。

代码清单 37-2 SDIO 命令初始化接口

```
1 typedef struct {  
2     uint32_t SDIO_Argument; // 命令参数  
3     uint32_t SDIO_CmdIndex; // 命令号  
4     uint32_t SDIO_Response; // 响应类型  
5     uint32_t SDIO_Wait;      // 等待使能  
6     uint32_t SDIO_CPSM;      // 命令路径状态机  
7 } SDIO_CmdInitTypeDef;
```

各个结构体成员介绍如下:

- (1) **SDIO_Argument**: 作为命令的一部分发送到卡的命令参数, 它设定 SDIO 参数寄存器(SDIO_ARG)的值。
- (2) **SDIO_CmdIndex**: 命令号选择, 它设定 SDIO 命令寄存器(SDIO_CMD)的 CMDINDEX 位的值。
- (3) **SDIO_Response**: 响应类型, SDIO 定义两个响应类型: 长响应和短响应。根据命令号选择对应的响应类型。SDIO 定义了四个 32 位的 SDIO 响应寄存器(SDIO_RESPx, x=1..4), 短响应只用到 SDIO_RESP1。
- (4) **SDIO_Wait**: 等待类型选择, 有三种状态可选, 一种是无等待状态, 超时检测功能启动; 一种是等待中断, 另外一种是等待传输完成。它设定 SDIO_CMD 寄存器的 WAITPEND 位和 WAITINT 位的值。
- (5) **SDIO_CPSM**: 命令路径状态机控制, 可选使能或禁用 CPSM。它设定 SDIO_CMD 寄存器的 CPSMEN 位的值。

37.8 SDIO 数据初始化结构体

SDIO 数据初始化结构体用于配置数据发送和接收参数, 比如传输超时、数据长度、传输模式等等。它被 SDIO_DataConfig 函数使用。

代码清单 37-3 SDIO 数据初始化结构体

```
1 typedef struct {  
2     uint32_t SDIO_DataTimeOut; // 数据传输超时  
3     uint32_t SDIO_DataLength;  // 数据长度  
4     uint32_t SDIO_DataBlockSize; // 数据块大小  
5     uint32_t SDIO_TransferDir;  // 数据传输方向  
6     uint32_t SDIO_TransferMode; // 数据传输模式  
7     uint32_t SDIO_DPSM;         // 数据路径状态机  
8 } SDIO_DataInitTypeDef;
```

各结构体成员介绍如下:

- (1) **SDIO_DataTimeOut**: 设置数据传输以卡总线时钟周期表示的超时周期, 它设定 SDIO 数据定时器寄存器(SDIO_DTIMER)的值。在 DPSM 进入 Wait_R 或繁忙状态后开始递减, 直到 0 还处于以上两种状态则将超时状态标志置 1。
- (2) **SDIO_DataLength**: 设置传输数据长度, 它设定 SDIO 数据长度寄存器(SDIO_DLEN)的值。
- (3) **SDIO_DataBlockSize**: 设置数据块大小, 有多种尺寸可选, 不同命令要求的数据块可能不同。它设定 SDIO 数据控制寄存器(SDIO_DCTRL)寄存器的 DBLOCKSIZE 位的值。
- (4) **SDIO_TransferDir**: 数据传输方向, 可选从主机到卡的写操作, 或从卡到主机的读操作。它设定 SDIO_DCTRL 寄存器的 DTDIR 位的值。

- (5) SDIO_TransferMode: 数据传输模式, 可选数据块或数据流模式。对于 SD 卡操作使用数据块类型。它设定 SDIO_DCTRL 寄存器的 DTMODE 位的值。
- (6) SDIO_DPSM: 数据路径状态机控制, 可选使能或禁用 DPSM。它设定 SDIO_DCTRL 寄存器的 DTEN 位的值。要实现数据传输都必须使能 SDIO_DPSM。

37.9 SD 卡读写测试实验

SD 卡广泛用于便携式设备上, 比如数码相机、手机、多媒体播放器等。对于嵌入式设备来说是一种重要的存储数据部件。类似与 SPI Flash 芯片数据操作, 可以直接进行读写, 也可以写入文件系统, 然后使用文件系统读写函数, 使用文件系统操作。本实验是进行 SD 卡最底层的数据读写操作, 直接使用 SDIO 对 SD 卡进行读写, 会损坏 SD 卡原本内容, 导致数据丢失, 实验前请注意备份 SD 卡的原内容。由于 SD 卡容量很大, 我们平时使用的 SD 卡都是已经包含有文件系统的, 一般不会使用本章的操作方式编写 SD 卡的应用, 但它是 SD 卡操作的基础, 对于原理学习是非常有必要的, 在它的基础上移植文件系统到 SD 卡的应用将在下一章讲解。

37.9.1 硬件设计

STM32 控制器的 SDIO 引脚是被设计固定不变的, 开发板设计采用四根数据线模式。对于命令线和数据线须需要加一个上拉电阻。

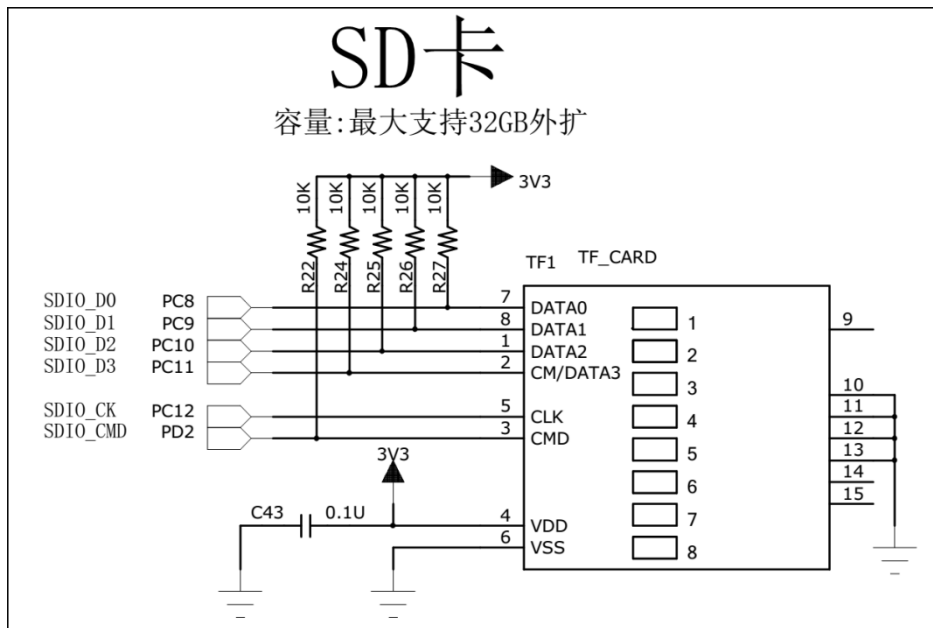


图 37-18 SD 卡硬件设计

37.9.2 软件设计

这里只讲解核心的部分代码, 有些变量的设置, 头文件的包含等没有全部罗列出来, 完整的代码请参考本章配套的工程。有了之前相关 SDIO 知识基础, 我们就可以着手开始编写 SD 卡驱动程序了, 根据之前内容, 可了解操作的大概流程:

- 初始化相关 GPIO 及 SDIO 外设;

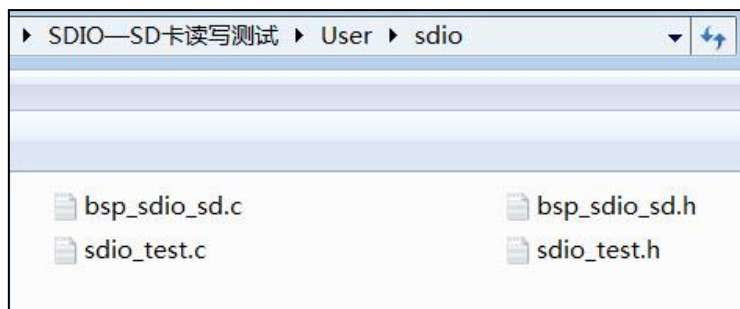


图 37-20 SD 卡驱动文件

1. GPIO 初始化和 DMA 配置

SDIO 用到 CLK 线、CMD 线和 4 根 DAT 线，使用之前必须初始化相关 GPIO，并设置为复用模式。SDIO 可以生成 DMA 请求，使用 DMA 传输可以提高数据传输效率。SDIO 可以设置为轮询模式或 DMA 传输模式，SD 卡驱动代码针对这两个模式做了区分处理，一般使用 DMA 传输模式，使用接下来代码分析都以 DMA 传输模式介绍。

GPIO 初始化和 DMA 配置这部分代码从 stm324x7i_eval.c 和 stm324x7i_eval.h 两个文件中移植而来。

DMA 及相关配置宏定义

代码清单 37-4 DMA 及相关配置宏定义

```
1 #define SDIO_FIFO_ADDRESS ((uint32_t)0x40012C80)
2 /**
3  * @brief SDIO Initialization Frequency (400KHz max)
4  */
5 #define SDIO_INIT_CLK_DIV ((uint8_t)0x76)
6 /**
7  * @brief SDIO Data Transfer Frequency (25MHz max)
8  */
9 #define SDIO_TRANSFER_CLK_DIV ((uint8_t)0x0)
10
11 #define SD_SDIO_DMA DMA2
12 #define SD_SDIO_DMA_CLK RCC_AHB1Periph_DMA2
13
14 #define SD_SDIO_DMA_STREAM3 3
15
16 #ifndef SD_SDIO_DMA_STREAM3
17 #define SD_SDIO_DMA_STREAM DMA2_Stream3
18 #define SD_SDIO_DMA_CHANNEL DMA_Channel_4
19 #define SD_SDIO_DMA_FLAG_FEIF DMA_FLAG_FEIF3
20 #define SD_SDIO_DMA_FLAG_DMEIF DMA_FLAG_DMEIF3
21 #define SD_SDIO_DMA_FLAG_TEIF DMA_FLAG_TEIF3
22 #define SD_SDIO_DMA_FLAG_HTIF DMA_FLAG_HTIF3
23 #define SD_SDIO_DMA_FLAG_TCIF DMA_FLAG_TCIF3
24 #define SD_SDIO_DMA_IRQn DMA2_Stream3_IRQn
25 #define SD_SDIO_DMA_IRQHANDLER DMA2_Stream3_IRQHandler
26 #elif defined SD_SDIO_DMA_STREAM6
27 #define SD_SDIO_DMA_STREAM DMA2_Stream6
28 #define SD_SDIO_DMA_CHANNEL DMA_Channel_4
29 #define SD_SDIO_DMA_FLAG_FEIF DMA_FLAG_FEIF6
30 #define SD_SDIO_DMA_FLAG_DMEIF DMA_FLAG_DMEIF6
31 #define SD_SDIO_DMA_FLAG_TEIF DMA_FLAG_TEIF6
32 #define SD_SDIO_DMA_FLAG_HTIF DMA_FLAG_HTIF6
33 #define SD_SDIO_DMA_FLAG_TCIF DMA_FLAG_TCIF6
34 #define SD_SDIO_DMA_IRQn DMA2_Stream6_IRQn
35 #define SD_SDIO_DMA_IRQHANDLER DMA2_Stream6_IRQHandler
```

```
36 #endif /* SD_SDIO_DMA_STREAM3 */
```

使用宏定义编程对程序在同系列而不同型号主控芯片移植起到很好的帮助，同时简化程序代码。数据 FIFO 起始地址可用于 DMA 传输地址；SDIOCLK 在卡识别模式和数据传输模式下一般是不同的，使用不同分频系数控制。SDIO 使用 DMA2 外设，可选择 stream3 和 stream6。

GPIO 初始化

代码清单 37-5 GPIO 初始化

```
1 void SD_LowLevel_Init(void)
2 {
3     GPIO_InitTypeDef GPIO_InitStructure;
4
5     /* GPIOC and GPIOD Periph clock enable */
6     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC | RCC_AHB1Periph_GPIOD,
7                             ENABLE);
8
9     GPIO_PinAFConfig(GPIOC, GPIO_PinSource8, GPIO_AF_SDIO);
10    GPIO_PinAFConfig(GPIOC, GPIO_PinSource9, GPIO_AF_SDIO);
11    GPIO_PinAFConfig(GPIOC, GPIO_PinSource10, GPIO_AF_SDIO);
12    GPIO_PinAFConfig(GPIOC, GPIO_PinSource11, GPIO_AF_SDIO);
13    GPIO_PinAFConfig(GPIOC, GPIO_PinSource12, GPIO_AF_SDIO);
14    GPIO_PinAFConfig(GPIOD, GPIO_PinSource2, GPIO_AF_SDIO);
15
16    /* Configure PC.08, PC.09, PC.10, PC.11 pins: D0, D1, D2, D3 pins */
17    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8 | GPIO_Pin_9 |
18                                   GPIO_Pin_10 | GPIO_Pin_11;
19    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
20    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
21    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
22    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
23    GPIO_Init(GPIOC, &GPIO_InitStructure);
24
25    /* Configure PD.02 CMD line */
26    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
27    GPIO_Init(GPIOD, &GPIO_InitStructure);
28
29    /* Configure PC.12 pin: CLK pin */
30    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12;
31    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
32    GPIO_Init(GPIOC, &GPIO_InitStructure);
33
34    /* Enable the SDIO APB2 Clock */
35    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SDIO, ENABLE);
36
37    /* Enable the DMA2 Clock */
38    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2, ENABLE);
39 }
```

由于 SDIO 对应的 IO 引脚都是固定的，所以这里没有使用宏定义方式给出，直接使用 GPIO 引脚，该函数初始化引脚之后还使能了 SDIO 和 DMA2 时钟。

DMA 传输配置

代码清单 37-6 DMA 传输配置

```
1 /* 配置使用 DMA 发送 */
2 void SD_LowLevel_DMA_TxConfig(uint32_t *BufferSRC, uint32_t BufferSize)
3 {
4     DMA_InitTypeDef SDDMA_InitStructure;
5
6     DMA_ClearFlag(SD_SDIO_DMA_STREAM, SD_SDIO_DMA_FLAG_FEIF |
7                  SD_SDIO_DMA_FLAG_DMEIF | SD_SDIO_DMA_FLAG_TEIF |
8                  SD_SDIO_DMA_FLAG_HTIF | SD_SDIO_DMA_FLAG_TCIF);
9 }
```

```
10  /* DMA2 Stream3 or Stream6 disable */
11  DMA_Cmd(SD_SDIO_DMA_STREAM, DISABLE);
12
13  /* DMA2 Stream3 or Stream6 Config */
14  DMA_DeInit(SD_SDIO_DMA_STREAM);
15
16  SDDMA_InitStructure.DMA_Channel = SD_SDIO_DMA_CHANNEL;
17  SDDMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)SDIO_FIFO_ADDRESS;
19  SDDMA_InitStructure.DMA_Memory0BaseAddr = (uint32_t)BufferSRC;
20  SDDMA_InitStructure.DMA_DIR = DMA_DIR_MemoryToPeripheral;
21  SDDMA_InitStructure.DMA_BufferSize = 1;
22  SDDMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
23  SDDMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
24  SDDMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
26  SDDMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;
27  SDDMA_InitStructure.DMA_Mode = DMA_Mode_Normal;
28  SDDMA_InitStructure.DMA_Priority = DMA_Priority_VeryHigh;
29  SDDMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Enable;
30  SDDMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;
31  SDDMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_INC4;
32  SDDMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_INC4;
33  DMA_Init(SD_SDIO_DMA_STREAM, &SDDMA_InitStructure);
34  DMA_ITConfig(SD_SDIO_DMA_STREAM, DMA_IT_TC, ENABLE);
35  DMA_FlowControllerConfig(SD_SDIO_DMA_STREAM, DMA_FlowCtrl_Peripheral);
36
37  /* DMA2 Stream3 or Stream6 enable */
38  DMA_Cmd(SD_SDIO_DMA_STREAM, ENABLE);
39
40 }
41
42 /* 配置使用 DMA 接收 */
43 void SD_LowLevel_DMA_RxConfig(uint32_t *BufferDST, uint32_t BufferSize)
44 {
45     DMA_InitTypeDef SDDMA_InitStructure;
46
47     DMA_ClearFlag(SD_SDIO_DMA_STREAM, SD_SDIO_DMA_FLAG_FEIF|
48                 SD_SDIO_DMA_FLAG_DMEIF|SD_SDIO_DMA_FLAG_TEIF|
49                 SD_SDIO_DMA_FLAG_HTIF|SD_SDIO_DMA_FLAG_TCIF);
50
51     /* DMA2 Stream3 or Stream6 disable */
52     DMA_Cmd(SD_SDIO_DMA_STREAM, DISABLE);
53
54     /* DMA2 Stream3 or Stream6 Config */
55     DMA_DeInit(SD_SDIO_DMA_STREAM);
56
57     SDDMA_InitStructure.DMA_Channel = SD_SDIO_DMA_CHANNEL;
58     SDDMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)SDIO_FIFO_ADDRESS;
60     SDDMA_InitStructure.DMA_Memory0BaseAddr = (uint32_t)BufferDST;
61     SDDMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;
62     SDDMA_InitStructure.DMA_BufferSize = 1;
63     SDDMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
64     SDDMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
65     SDDMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
67     SDDMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;
68     SDDMA_InitStructure.DMA_Mode = DMA_Mode_Normal;
69     SDDMA_InitStructure.DMA_Priority = DMA_Priority_VeryHigh;
70     SDDMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Enable;
71     SDDMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;
72     SDDMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_INC4;
73     SDDMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_INC4;
74     DMA_Init(SD_SDIO_DMA_STREAM, &SDDMA_InitStructure);
75     DMA_ITConfig(SD_SDIO_DMA_STREAM, DMA_IT_TC, ENABLE);
76     DMA_FlowControllerConfig(SD_SDIO_DMA_STREAM, DMA_FlowCtrl_Peripheral);
77
78     /* DMA2 Stream3 or Stream6 enable */
79     DMA_Cmd(SD_SDIO_DMA_STREAM, ENABLE);
80 }
```

SD_LowLevel_DMA_TxConfig 函数用于配置 DMA 的 SDIO 发送请求参数,并指定发送存储器地址和大小。SD_LowLevel_DMA_RxConfig 函数用于配置 DMA 的 SDIO 接收请求参数,并指定接收存储器地址和大小。实际上,仔细分析代码可以发现 BufferSize 参数在这里是没有被用到,一般在使用 DMA 参数时都是指定传输的数量,DMA 控制器在传输指定的数量后自动停止,但对于 SD 卡来说,可以生成硬件控制流,在传输完目标数量数据后即控制传输停止,所以这里调用 DMA_FlowControllerConfig 函数使能 SD 卡作为 DMA 传输停止的控制,这样 BufferSize 参数无需用到也可以正确传输。对于 DMA 相关配置可以参考 DMA 章节内容。

2. 相关类型定义

打开 bsp_sdio_sd.h 文件可以发现非常多的枚举类型定义、结构体类型定义以及宏定义,把所有的定义在这里罗列出来肯定是不现实的,此处简要介绍如下:

- ❑ 枚举类型定义:有 SD_Error、SDTransferState 和 SDCardState 三个。SD_Error 是列举了控制器可能出现的错误、比如 CRC 校验错误、CRC 校验错误、通信等待超时、FIFO 上溢或下溢、擦除命令错误等等。这些错误类型部分是控制器系统寄存器的标志位,部分是通过命令的响应内容得到的。SDTransferState 定义了 SDIO 传输状态,有传输正常状态、传输忙状态和传输错误状态。SDCardState 定义卡的当前状态,比如准备状态、识别状态、待机状态、传输状态等等,具体状态转换过程参考图 37-9 和图 37-10。
- ❑ 结构体类型定义:有 SD_CSD、SD_CID、SD_CardStatus 以及 SD_CardInfo。
SD_CSD 定义了 SD 卡的特定数据(CSD)寄存器位,一般提供 R2 类型的响应可以获取得到 CSD 寄存器内容。SD_CID 结构体类似 SD_CSD 结构体,它定义 SD 卡 CID 寄存器内容,也是通过 R2 响应类型获取得到。SD_CardStatus 结构体定义了 SD 卡状态,有数据宽度、卡类型、速度等级、擦除宽度、传输偏移地址等等 SD 卡状态。SD_CardInfo 结构体定义了 SD 卡信息,包括了 SD_CSD 类型和 SD_CID 类型成员,还有定义了卡容量、卡块大小、卡相对地址 RCA 和卡类型成员。
- ❑ 宏定义内容:包含有命令号定义、SDIO 传输方式、SD 卡插入状态以及 SD 卡类型定义。参考表 37-2 列举了描述了部分命令,文件中为每个命令号定义一个宏,比如将复位 CMD0 定义为 SD_CMD_GO_IDLE_STATE,这与表 37-2 中缩写部分是类似的,所以熟悉命名用途可以更好理解 SD 卡操作过程。SDIO 数据传输可以选择是否使用 DMA 传输,SD_DMA_MODE 宏定义选择 DMA 传输,SD_POLLING_MODE 使用普通扫描和传输,只能二选一使用。为提高系统性能,一般使用 DMA 传输模式,ST 官方的 SD 卡驱动对这两个方式做了区分出来,下面对 SD 卡操作都是以 DMA 传输模式为例讲解。接下来还定义了检测 SD 卡是否正确插入的宏,ST 官方的 SD 卡驱动是以一个输入引脚电平判断 SD 卡是否正确插入,这里我们不使用,把引脚定义去掉(不然编译出错),保留 SD_PRESENT 和 SD_NOT_PRESENT 两个宏定义。最后定义 SD 卡具体的类型,有 V1.1 版本标准卡、V2.0 版本标准卡、高容量 SD 卡以及其他类型卡,前面三个是常用的类型。

在 bsp_sdio_sd.c 文件也有部分宏定义，这部分宏定义只能在该文件中使用。这部分宏定义包括命令超时时间定义、OCR 寄存器位掩码、R6 响应位掩码等等，这些定义更多是为提取特定响应位内容而设计的掩码。

因为类型定义和宏定义内容没有在本文中列举出来，读者有必要使用 KEIL 工具打开本章配套例程理解清楚。同时了解 bsp_sdio_sd.c 文件中定义多个不同类型变量。

接下来我们就开始根据 SD 卡识别过程和数据传输过程理解 SD 卡驱动函数代码。这部分代码内容也是非常庞大，不可能全部在文档中全部列出，对于部分函数只介绍其功能。

3. SD 卡初始化

SD 卡初始化过程主要是卡识别和相关 SD 卡状态获取。整个初始化函数可以实现图 37-21 中的功能。

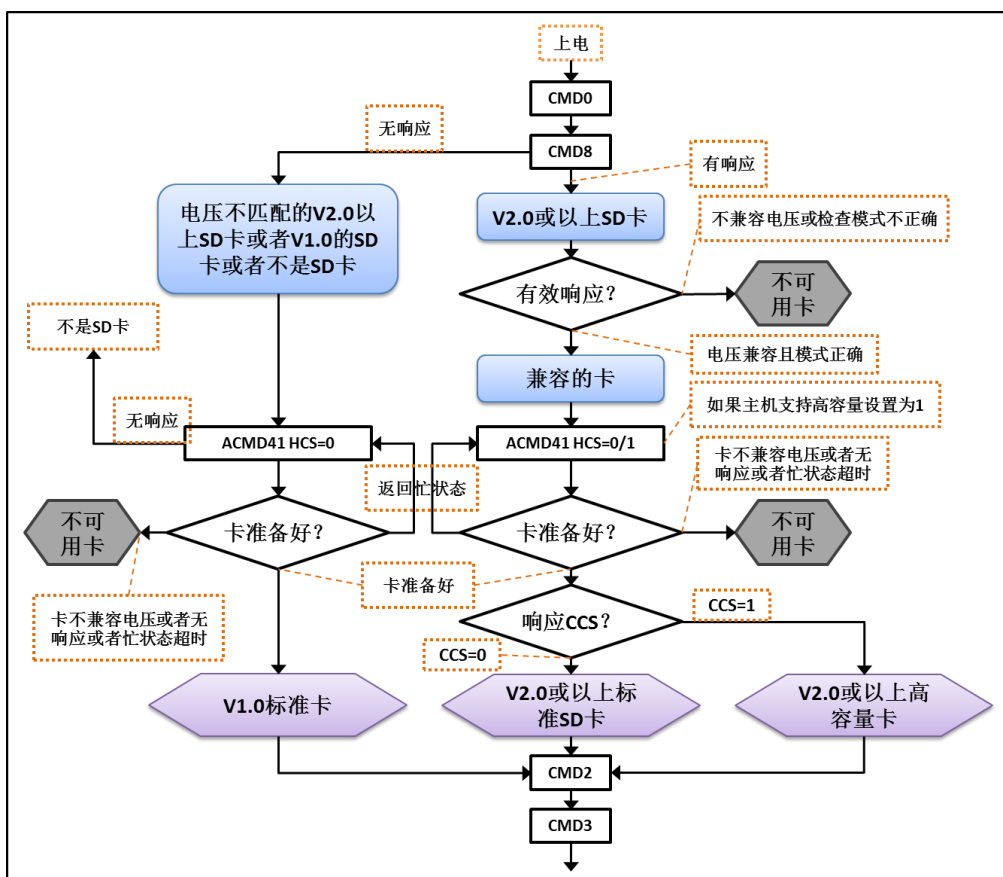


图 37-21 SD 卡初始化和识别流程

SD 卡初始化函数

代码清单 37-7 SD_Init 函数

```

1 SD_Error SD_Init(void)
2 {
3     __IO SD_Error errorstatus = SD_OK;
4
5     /*****配置 SDIO 中断 DMA 中断*****/
6     NVIC_InitTypeDef NVIC_InitStructure;
7
8     // Configure the NVIC Preemption Priority Bits
9     NVIC_PriorityGroupConfig (NVIC_PriorityGroup_1);

```

```
10     NVIC_InitStructure.NVIC_IRQChannel = SDIO_IRQn;
11     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
12     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
13     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
14     NVIC_Init (&NVIC_InitStructure);
15     NVIC_InitStructure.NVIC_IRQChannel = SD_SDIO_DMA_IRQn;
16     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
17     NVIC_Init (&NVIC_InitStructure);
18     /*****/
19
20     /* SDIO Peripheral Low Level Init */
21     SD_LowLevel_Init();
22
23     SDIO_DeInit();
24
25     errorstatus = SD_PowerON();
26
27     if (errorstatus != SD_OK) {
28         /*!< CMD Response TimeOut (wait for CMDSENT flag) */
29         return (errorstatus);
30     }
31
32     errorstatus = SD_InitializeCards();
33
34     if (errorstatus != SD_OK) {
35         /*!< CMD Response TimeOut (wait for CMDSENT flag) */
36         return (errorstatus);
37     }
38
39     /*!< Configure the SDIO peripheral */
40     /*!< SDIO_CK = SDIOCLK / (SDIO_TRANSFER_CLK_DIV + 2) */
41     /*!< on STM32F4xx devices, SDIOCLK is fixed to 48MHz */
42     SDIO_InitStructure.SDIO_ClockDiv = SDIO_TRANSFER_CLK_DIV;
43     SDIO_InitStructure.SDIO_ClockEdge = SDIO_ClockEdge_Rising;
44     SDIO_InitStructure.SDIO_ClockBypass = SDIO_ClockBypass_Disable;
45     SDIO_InitStructure.SDIO_ClockPowerSave = SDIO_ClockPowerSave_Disable;
46     SDIO_InitStructure.SDIO_BusWide = SDIO_BusWide_1b;
47     SDIO_InitStructure.SDIO_HardwareFlowControl = SDIO_HardwareFlowControl_Disable;
48     SDIO_Init(&SDIO_InitStructure);
49
50
51     /*----- Read CSD/CID MSD registers -----*/
52     errorstatus = SD_GetCardInfo(&SDCardInfo);
53
54     if (errorstatus == SD_OK) {
55         /*----- Select Card -----*/
56         errorstatus = SD_SelectDeselect((uint32_t) (SDCardInfo.RCA << 16));
57     }
58
59     if (errorstatus == SD_OK) {
60         errorstatus = SD_EnableWideBusOperation(SDIO_BusWide_4b);
61     }
62
63     return (errorstatus);
64 }
```

该函数的部分执行流程如下：

- (1) 配置 NVIC，SD 卡通信用到 SDIO 中断，如果用到 DMA 传输还需要配置 DMA 中断。注意中断服务函数不是定义在 stm32f4xx_it.c 文件的，是直接定义在 bsp_sdio_sd.c 文件中，中断服务函数定义在哪个文件问题都不大，只要定义正确就可以的，编译器会自动寻找。
- (2) 执行 SD_LowLevel_Init 函数，其功能是对底层 SDIO 引脚进行初始化以及开启相关时钟，该函数在之前已经讲解。

- (3) SDIO_DeInit 函数用于解除初始化 SDIO 接口，它只是简单调用 SD_LowLevel_DeInit 函数。而 SD_LowLevel_DeInit 函数是与 SD_LowLevel_Init 函数相反功能，关闭相关时钟，关闭 SDIO 电源，让 SDIO 接近上电复位状态。恢复复位状态后再进行相关配置，可以防止部分没有配置的参数采用非默认值而导致错误，这是 ST 官方驱动常用的一种初始化方式。
- (4) 调用 SD_PowerON 函数，它用于查询卡的工作电压和时钟控制配置，并返回 SD_Error 类型错误，该函数是整个 SD 识别精髓，有必要详细分析。

SD_POWERON 函数

代码清单 37-8 SD_POWERON 函数

```
1 SD_Error SD_PowerON(void)
2 {
3     __IO SD_Error errorstatus = SD_OK;
4     uint32_t response = 0, count = 0, validvoltage = 0;
5     uint32_t SDType = SD_STD_CAPACITY;
6     /*!< Power ON Sequence -----*/
7     SDIO_InitStructure.SDIO_ClockDiv = SDIO_INIT_CLK_DIV;
8     SDIO_InitStructure.SDIO_ClockEdge = SDIO_ClockEdge_Rising;
9     SDIO_InitStructure.SDIO_ClockBypass = SDIO_ClockBypass_Disable;
10    SDIO_InitStructure.SDIO_ClockPowerSave = SDIO_ClockPowerSave_Disable;
11    SDIO_InitStructure.SDIO_BusWide = SDIO_BusWide_1b;
12    SDIO_InitStructure.SDIO_HardwareFlowControl = SDIO_HardwareFlowControl_Disable;
13    SDIO_Init(&SDIO_InitStructure);
14    /*!< Set Power State to ON */
15    SDIO_SetPowerState(SDIO_PowerState_ON);
16    /*!< Enable SDIO Clock */
17    SDIO_ClockCmd(ENABLE);
18    /*!< CMD0: GO_IDLE_STATE -----*/
19    SDIO_CmdInitStructure.SDIO_Argument = 0x0;
20    SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_GO_IDLE_STATE;
21    SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_No;
22    SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
23    SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
24    SDIO_SendCommand(&SDIO_CmdInitStructure);
25    errorstatus = CmdError();
26    if (errorstatus != SD_OK) {
27        return (errorstatus);
28    }
29    /*!< CMD8: SEND_IF_COND -----*/
30    SDIO_CmdInitStructure.SDIO_Argument = SD_CHECK_PATTERN;
31    SDIO_CmdInitStructure.SDIO_CmdIndex = SDIO_SEND_IF_COND;
32    SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
33    SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
34    SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
35    SDIO_SendCommand(&SDIO_CmdInitStructure);
36    errorstatus = CmdResp7Error();
37    if (errorstatus == SD_OK) {
38        CardType = SDIO_STD_CAPACITY_SD_CARD_V2_0; /*!< SD Card 2.0 */
39        SDType = SD_HIGH_CAPACITY;
40    } else {
41        /*!< CMD55 */
42        SDIO_CmdInitStructure.SDIO_Argument = 0x00;
43        SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_APP_CMD;
44        SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
45        SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
46        SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
47        SDIO_SendCommand(&SDIO_CmdInitStructure);
48        errorstatus = CmdResp1Error(SD_CMD_APP_CMD);
49    }
50    /*!< CMD55 */
51    SDIO_CmdInitStructure.SDIO_Argument = 0x00;
52    SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_APP_CMD;
```

```
54     SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
55     SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
56     SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
57     SDIO_SendCommand(&SDIO_CmdInitStructure);
58     errorstatus = CmdResp1Error(SD_CMD_APP_CMD);
59     /*!< If errorstatus is Command Timeout, it is a MMC card */
60     /*!< If errorstatus is SD_OK it is a SD card: SD card 2.0
61         (voltage range mismatch)or SD card 1.x */
62     if (errorstatus == SD_OK) {
63         /*!< SD CARD */
64         /*!< Send ACMD41 SD_APP_OP_COND with Argument 0x80100000 */
65         while ((!validvoltage) && (count < SD_MAX_VOLT_TRIAL)) {
66             /*!< SEND CMD55 APP_CMD with RCA as 0 */
67             SDIO_CmdInitStructure.SDIO_Argument = 0x00;
68             SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_APP_CMD;
69             SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
70             SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
71             SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
72             SDIO_SendCommand(&SDIO_CmdInitStructure);
73             errorstatus = CmdResp1Error(SD_CMD_APP_CMD);
74             if (errorstatus != SD_OK) {
75                 return (errorstatus);
76             }
77             SDIO_CmdInitStructure.SDIO_Argument = SD_VOLTAGE_WINDOW_SD|SDType;
78             SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_SD_APP_OP_COND;
79             SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
80             SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
81             SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
82             SDIO_SendCommand(&SDIO_CmdInitStructure);
83             errorstatus = CmdResp3Error();
84             if (errorstatus != SD_OK) {
85                 return (errorstatus);
86             }
87             response = SDIO_GetResponse(SDIO_RESP1);
88             validvoltage = (((response >> 31) == 1) ? 1 : 0);
89             count++;
90         }
91         if (count >= SD_MAX_VOLT_TRIAL) {
92             errorstatus = SD_INVALID_VOLTRANGE;
93             return (errorstatus);
94         }
95         if (response &= SD_HIGH_CAPACITY) {
96             CardType = SDIO_HIGH_CAPACITY_SD_CARD;
97         }
98     } /*!< else MMC Card */
99     return (errorstatus);
100 }
```

SD_POWERON 函数执行流程如下:

- (1) 配置 SDIO_InitStructure 结构体变量成员并调用 SDIO_Init 库函数完成 SDIO 外设的基本配置, 注意此处的 SDIO 时钟分频, 由于处于卡识别阶段, 其时钟不能超过 400KHz。
- (2) 调用 SDIO_SetPowerState 函数控制 SDIO 的电源状态, 给 SDIO 提供电源, 并调用 ClockCmd 库函数使能 SDIO 时钟。
- (3) 发送命令给 SD 卡, 首先发送 CMD0, 复位所有 SD 卡, CMD0 命令无需响应, 所以调用 CmdError 函数检测错误即可。CmdError 函数用于无需响应的命令发送检测, 带有等待超时检测功能, 它通过不断检测 SDIO_STA 寄存器的 CMDSSENT 位即可知道命令发送成功与否。如果遇到超时错误则直接退出 SD_PowerON 函数。若无错误则执行下面程序。

- (4) 发送 CMD8 命令，检测 SD 卡支持的操作条件，主要就是电压匹配，CMD8 的响应类型是 R7，使用 CmdResp7Error 函数可获取到 R7 响应结果，它是通过检测 SDIO_STA 寄存器相关位完成的，并具有等待超时检测功能。如果 CmdResp7Error 函数返回值为 SD_OK，即 CMD8 有响应，可以判定 SD 卡为 V2.0 及以上的高容量 SD 卡，如果没有响应可能是 V1.1 版本卡或者是不可用卡。
- (5) 使用 ACMD41 命令判断卡的具体类型。在发送 ACMD41 之前必须先发送 CMD55，CMD55 命令的响应类型的 R1。如果 CMD55 命令都没有响应说明是 MMC 卡或不可用卡。在正确发送 CMD55 之后就可以发送 ACMD41，并根据响应判断卡类型，ACMD41 的响应号为 R3，CmdResp3Error 函数用于检测命令正确发送并带有超时检测功能，但并不具备响应内容接收功能，需要在判定命令正确发送之后调用 SDIO_GetResponse 函数才能获取响应的内容。实际上，在有响应时，SDIO 外设会自动把响应存放在 SDIO_RESPx 寄存器中，SDIO_GetResponse 函数只是根据形参返回对响应寄存器的值。通过判定响应内容值即可确定 SD 卡类型。
- (6) 执行 SD_PowerON 函数无错误后就已经确定了 SD 卡类型，并说明卡和主机电压是匹配的，SD 卡处于卡识别模式下的准备状态。退出 SD_PowerON 函数返回 SD_Init 函数，执行接下来代码。判断执行 SD_PowerON 函数无错误后，执行下面的 SD_InitializeCards 函数进行与 SD 卡相关的初始化，使得卡进入数据传输模式下的待机模式。

SD_InitializeCards 函数

代码清单 37-9 SD_InitializeCards 函数

```
1 SD_Error SD_InitializeCards(void)
2 {
3     SD_Error errorstatus = SD_OK;
4     uint16_t rca = 0x01;
5     if (SDIO_GetPowerState() == SDIO_PowerState_OFF) {
6         errorstatus = SD_REQUEST_NOT_APPLICABLE;
7         return (errorstatus);
8     }
9     if (SDIO_SECURE_DIGITAL_IO_CARD != CardType) {
10        /*!< Send CMD2 ALL_SEND_CID */
11        SDIO_CmdInitStructure.SDIO_Argument = 0x0;
12        SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_ALL_SEND_CID;
13        SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Long;
14        SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
15        SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
16        SDIO_SendCommand(&SDIO_CmdInitStructure);
17        errorstatus = CmdResp2Error();
18        if (SD_OK != errorstatus) {
19            return (errorstatus);
20        }
21        CID_Tab[0] = SDIO_GetResponse(SDIO_RESP1);
22        CID_Tab[1] = SDIO_GetResponse(SDIO_RESP2);
23        CID_Tab[2] = SDIO_GetResponse(SDIO_RESP3);
24        CID_Tab[3] = SDIO_GetResponse(SDIO_RESP4);
25    }
26    if ( (SDIO_STD_CAPACITY_SD_CARD_V1_1==CardType) ||
27        (SDIO_STD_CAPACITY_SD_CARD_V2_0==CardType) ||
28        (SDIO_SECURE_DIGITAL_IO_COMBO_CARD == CardType) ||
29        (SDIO_HIGH_CAPACITY_SD_CARD == CardType) ) {
30        /*!< Send CMD3 SET_REL_ADDR with argument 0 */
31        /*!< SD Card publishes its RCA. */
32        SDIO_CmdInitStructure.SDIO_Argument = 0x00;
```

```
33     SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_SET_REL_ADDR;
34     SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
35     SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
36     SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
37     SDIO_SendCommand(&SDIO_CmdInitStructure);
38     errorstatus = CmdResp6Error(SD_CMD_SET_REL_ADDR, &rca);
39     if (SD_OK != errorstatus) {
40         return (errorstatus);
41     }
42 }
43 if (SDIO_SECURE_DIGITAL_IO_CARD != CardType) {
44     RCA = rca;
45     /*!< Send CMD9 SEND_CSD with argument as card's RCA */
46     SDIO_CmdInitStructure.SDIO_Argument = (uint32_t)(rca << 16);
47     SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_SEND_CSD;
48     SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Long;
49     SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
50     SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
51     SDIO_SendCommand(&SDIO_CmdInitStructure);
52     errorstatus = CmdResp2Error();
53     if (SD_OK != errorstatus) {
54         return (errorstatus);
55     }
56     CSD_Tab[0] = SDIO_GetResponse(SDIO_RESP1);
57     CSD_Tab[1] = SDIO_GetResponse(SDIO_RESP2);
58     CSD_Tab[2] = SDIO_GetResponse(SDIO_RESP3);
59     CSD_Tab[3] = SDIO_GetResponse(SDIO_RESP4);
60 }
61 errorstatus = SD_OK; /*!< All cards get intialized */
62 return (errorstatus);
63 }
```

SD_InitializeCards 函数执行流程如下：

- (1) 判断 SDIO 电源是否启动，如果没有启动电源返回错误。
- (2) SD 卡不是 SD I/O 卡时会进入 if 判断，执行发送 CMD2，CMD2 是用于通知所有卡通过 CMD 线返回 CID 值，执行 CMD2 发送之后就可以使用 CmdResp2Error 函数获取 CMD2 命令发送情况，发送无错误后即可使用 SDIO_GetResponse 函数获取响应内容，它是个长响应，我们把 CMD2 响应内容存放在 CID_Tab 数组内。
- (3) 发送 CMD2 之后紧接着就发送 CMD3，用于指示 SD 卡自行推荐 RCA 地址，CMD3 的响应为 R6 类型，CmdResp6Error 函数用于检查 R6 响应错误，它有两个形参，一个是命令号，这里为 CMD3，另外一个为 RCA 数据指针，这里使用 rca 变量的地址赋值给它，使得在 CMD3 正确响应之后 rca 变量即存放 SD 卡的 RCA。R6 响应还有一部分位用于指示卡的状态，CmdResp6Error 函数通用会对每个错误位进行必要的检测，如果发现错误存在则直接返回对应错误类型。执行完 CmdResp6Error 函数之后返回到 SD_InitializeCards 函数中，如果判断无错误说明此刻 SD 卡已经处于数据传输模式。
- (4) 发送 CMD9 给指定 RCA 的 SD 卡使其发送返回其 CSD 寄存器内容，这里的 RCA 就是在 CmdResp6Error 函数获取到的 rca。最后把响应内容存放在 CSD_Tab 数组中。

执行 SD_InitializeCards 函数无错误后 SD 卡就已经处于数据传输模式下的待机状态，退出 SD_InitializeCards 后会返回前面的 SD_Init 函数，执行接下来代码，以下是 SD_Init 函数的后续执行过程：

- (1) 重新配置 SDIO 外设，提高时钟频率，之前的卡识别模式都设定 CMD 线时钟为小于 400KHz，进入数据传输模式可以把时钟设置为小于 25MHz，以便提高数据传输速率。
- (2) 调用 SD_GetCardInfo 函数获取 SD 卡信息，它需要一个指向 SD_CardInfo 类型变量地址的指针形参，这里赋值为 SDCardInfo 变量的地址。SD 卡信息主要是 CID 和 CSD 寄存器内容，这两个寄存器内容在 SD_InitializeCards 函数中都完成读取过程并将其分别存放在 CID_Tab 数组和 CSD_Tab 数组中，所以 SD_GetCardInfo 函数只是简单的把这两个数组内容整合复制到 SDCardInfo 变量对应成员内。正确执行 SD_GetCardInfo 函数后，SDCardInfo 变量就存放了 SD 卡的很多状态信息，这在之后应用中使用频率是很高的。
- (3) 调用 SD_SelectDeselect 函数用于选择特定 RCA 的 SD 卡，它实际是向 SD 卡发送 CMD7。执行之后，卡就从待机状态转变为传输模式，可以说数据传输已经是万事俱备了。
- (4) 扩展数据线宽度，之前的所有操作都是使用一根数据线传输完成的，使用 4 根数据线可以提高传输性能，调用可以设置数据线宽度，函数只有一个形参，用于指定数据线宽度。在 SD_EnableWideBusOperation 函数中，调用了 SDEnWideBus 函数使能使用宽数据线，然后传输 SDIO_InitTypeDef 类型变量并使用 SDIO_Init 函数完成使用 4 根数据线配置。

至此，SD_Init 函数已经全部执行完成。如果程序可以正确执行，接下来就可以进行 SD 卡读写以及擦除等操作。虽然 bsp_sdio_sd.c 文件看起来非常长，但在 SD_Init 函数分析过程就已经涉及到它差不多一半内容了，另外一半内容主要就是读、写或擦除相关函数。

4. SD 卡数据操作

SD 卡数据操作一般包括数据读取、数据写入以及存储区擦除。数据读取和写入都可以分为单块操作和多块操作。

擦除函数

代码清单 37-10 SD_Erase 函数

```
1 SD_Error SD_Erase(uint64_t startaddr, uint64_t endaddr)
2 {
3     SD_Error errorstatus = SD_OK;
4     uint32_t delay = 0;
5     __IO uint32_t maxdelay = 0;
6     uint8_t cardstate = 0;
7     /*!< Check if the card coomnd class supports erase command */
8     if (((CSD_Tab[1] >> 20) & SD_CCCC_ERASE) == 0) {
9         errorstatus = SD_REQUEST_NOT_APPLICABLE;
10        return (errorstatus);
11    }
12    maxdelay = 120000 / ((SDIO->CLKCR & 0xFF) + 2);
13    if (SDIO_GetResponse(SDIO_RESP1) & SD_CARD_LOCKED) {
14        errorstatus = SD_LOCK_UNLOCK_FAILED;
15        return (errorstatus);
16    }
17    if (CardType == SDIO_HIGH_CAPACITY_SD_CARD) {
18        startaddr /= 512;
19        endaddr /= 512;
20    }
```



```
21  /*!< ERASE_GROUP_START (CMD32) and erase_group_end(CMD33) */
22  if ( (SDIO_STD_CAPACITY_SD_CARD_V1_1 == CardType) ||
23        (SDIO_STD_CAPACITY_SD_CARD_V2_0 == CardType) ||
24        (SDIO_HIGH_CAPACITY_SD_CARD == CardType) ) {
25      /*!< Send CMD32 SD_ERASE_GRP_START with argument as addr */
26      SDIO_CmdInitStructure.SDIO_Argument = (uint32_t)startaddr;
27      SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_SD_ERASE_GRP_START;
28      SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
29      SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
30      SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
31      SDIO_SendCommand(&SDIO_CmdInitStructure);
32      errorstatus = CmdResplError(SD_CMD_SD_ERASE_GRP_START);
33      if (errorstatus != SD_OK) {
34          return (errorstatus);
35      }
36      /*!< Send CMD33 SD_ERASE_GRP_END with argument as addr */
37      SDIO_CmdInitStructure.SDIO_Argument = (uint32_t)endaddr;
38      SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_SD_ERASE_GRP_END;
39      SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
40      SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
41      SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
42      SDIO_SendCommand(&SDIO_CmdInitStructure);
43      errorstatus = CmdResplError(SD_CMD_SD_ERASE_GRP_END);
44      if (errorstatus != SD_OK) {
45          return (errorstatus);
46      }
47  }
48  /*!< Send CMD38 ERASE */
49  SDIO_CmdInitStructure.SDIO_Argument = 0;
50  SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_ERASE;
51  SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
52  SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
53  SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
54  SDIO_SendCommand(&SDIO_CmdInitStructure);
55  errorstatus = CmdResplError(SD_CMD_ERASE);
56  if (errorstatus != SD_OK) {
57      return (errorstatus);
58  }
59  for (delay = 0; delay < maxdelay; delay++) {
60  }
61  /*!< Wait till the card is in programming state */
62  errorstatus = IsCardProgramming(&cardstate);
63  delay = SD_DATATIMEOUT;
64  while ((delay > 0) && (errorstatus == SD_OK) &&
65        ((SD_CARD_PROGRAMMING == cardstate) || (SD_CARD_RECEIVING == cardstate))) {
66      errorstatus = IsCardProgramming(&cardstate);
67      delay--;
68  }
69  return (errorstatus);
70 }
```

SD_Erase 函数用于擦除 SD 卡指定地址范围内的数据。该函数接收两个参数，一个是擦除的起始地址，另外一个为擦除的结束地址。对于大容量 SD 卡都是以块大小为 512 字节进行擦除的，所以保证字节对齐是程序员的责任。SD_Erase 函数的执行流程如下：

- (1) 检查 SD 卡是否支持擦除功能，如果不支持则直接返回错误。为保证擦除指令正常进行，要求主机一个遵循下面的命令序列发送指令：CMD32->CMD33->CMD38。如果发送顺序不对，SD 卡会设置 ERASE_SEQ_ERROR 位到状态寄存器。
- (2) SD_Erase 函数发送 CMD32 指令用于设定擦除块开始地址，在执行无错误后发送 CMD33 设置擦除块的结束地址。

- (3) 发送擦除命令 CMD38，使得 SD 卡进行擦除操作。SD 卡擦除操作由 SD 卡内部控制完成，不同卡擦除后是 0xff 还是 0x00 由厂家决定。擦除操作需要花费一定时间，这段时间不能对 SD 卡进行其他操作。
- (4) 通过 IsCardProgramming 函数可以检测 SD 卡是否处于编程状态(即卡内部的擦写状态)，需要确保 SD 卡擦除完成才退出 SD_Erase 函数。IsCardProgramming 函数先通过发送 CMD13 命令 SD 卡发送它的状态寄存器内容，并对响应内容进行分析得出当前 SD 卡的状态以及可能发送的错误。

数据写入操作

数据写入可分为单块数据写入和多块数据写入，这里只分析单块数据写入，多块的与之类似。SD 卡数据写入之前并没有硬性要求擦除写入块，这与 SPI Flash 芯片写入是不同的。ST 官方的 SD 卡写入函数包括扫描查询方式和 DMA 传输方式，我们这里只介绍 DMA 传输模式。

代码清单 37-11 SD_WriteBlock 函数

```
1 SD_Error SD_WriteBlock(uint8_t *writebuff, uint64_t WriteAddr,
2                          uint16_t BlockSize)
3 {
4     SD_Error errorstatus = SD_OK;
5     TransferError = SD_OK;
6     TransferEnd = 0;
7     StopCondition = 0;
8     SDIO->DCTRL = 0x0;
9     #if defined (SD_DMA_MODE)
10    SDIO_ITConfig(SDIO_IT_DCRCFail | SDIO_IT_DTIMEOUT | SDIO_IT_DATAEND |
11                  SDIO_IT_RXOVERR | SDIO_IT_STBITERR, ENABLE);
12    SD_LowLevel_DMA_TxConfig((uint32_t *)writebuff, BlockSize);
13    SDIO_DMACmd(ENABLE);
14 #endif
15    if (CardType == SDIO_HIGH_CAPACITY_SD_CARD) {
16        BlockSize = 512;
17        WriteAddr /= 512;
18    }
19    /* Set Block Size for Card */
20    SDIO_CmdInitStructure.SDIO_Argument = (uint32_t) BlockSize;
21    SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_SET_BLOCKLEN;
22    SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
23    SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
24    SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
25    SDIO_SendCommand(&SDIO_CmdInitStructure);
26    errorstatus = CmdResplError(SD_CMD_SET_BLOCKLEN);
27    if (SD_OK != errorstatus) {
28        return (errorstatus);
29    }
30    /*!< Send CMD24 WRITE_SINGLE_BLOCK */
31    SDIO_CmdInitStructure.SDIO_Argument = (uint32_t)WriteAddr;
32    SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_WRITE_SINGLE_BLOCK;
33    SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
34    SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
35    SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
36    SDIO_SendCommand(&SDIO_CmdInitStructure);
37    errorstatus = CmdResplError(SD_CMD_WRITE_SINGLE_BLOCK);
38
39    if (errorstatus != SD_OK) {
40        return (errorstatus);
41    }
42    SDIO_DataInitStructure.SDIO_DataTimeOut = SD_DATATIMEOUT;
43    SDIO_DataInitStructure.SDIO_DataLength = BlockSize;
44    SDIO_DataInitStructure.SDIO_DataBlockSize = (uint32_t) 9 << 4;
```

```
45     SDIO_DataInitStructure.SDIO_TransferDir = SDIO_TransferDir_ToCard;
46     SDIO_DataInitStructure.SDIO_TransferMode = SDIO_TransferMode_Block;
47     SDIO_DataInitStructure.SDIO_DPSM = SDIO_DPSM_Enable;
48     SDIO_DataConfig(&SDIO_DataInitStructure);
49     return (errorstatus);
50 }
```

SD_WriteBlock 函数用于向指定的目标地址写入一个块的数据，它有三个形参，分别为指向待写入数据的首地址的指针变量、目标写入地址和块大小。块大小一般都设置为 512 字节。SD_WriteBlock 写入函数的执行流程如下：

- (1) SD_WriteBlock 函数开始将 SDIO 数据控制寄存器 (SDIO_DCTRL) 清理，复位之前的传输设置。
- (2) 来调用 SDIO_ITConfig 函数使能相关中断，包括数据 CRC 失败中断、数据超时中断、数据结束中断等等。
- (3) 调用 SD_LowLevel_DMA_TxConfig 函数，配置使能 SDIO 数据向 SD 卡的数据传输的 DMA 请求，该函数可以参考代码清单 37-6。为使 SDIO 发送 DMA 请求，需要调用 SDIO_DMAMCmd 函数使能。对于高容量的 SD 卡要求块大小必须为 512 字节，程序员有责任保证数据写入地址与块大小的字节对齐问题。
- (4) 对 SD 卡进行数据读写之前，都必须发送 CMD16 指定块的大小，对于标准卡，要写入 BlockSize 长度字节的块；对于 SDHC 卡，写入 512 字节的块。接下来就可以发送块写入命令 CMD24 通知 SD 卡要进行数据写入操作，并指定待写入数据的目标地址。
- (5) 利用 SDIO_DataInitTypeDef 结构体类型变量配置数据传输的超时、块数量、数据块大小、数据传输方向等参数并使用 SDIO_DataConfig 函数完成数据传输环境配置。

执行完以上代码后，SDIO 外设会自动生成 DMA 发送请求，将指定数据使用 DMA 传输写入到 SD 卡内。

写入操作等待函数

SD_WaitWriteOperation 函数用于检测和等待数据写入完成，在调用数据写入函数之后一般都需要调用，SD_WaitWriteOperation 函数不仅使用于单块写入函数也适用于多块写入函数。

代码清单 37-12 SD_WaitWriteOperation 函数

```
1 SD_Error SD_WaitWriteOperation(void)
2 {
3     SD_Error errorstatus = SD_OK;
4     uint32_t timeout;
5
6     timeout = SD_DATATIMEOUT;
7     while ( (DMAEndOfTransfer == 0x00) && (TransferEnd == 0) &&
8             (TransferError == SD_OK) && (timeout > 0) ) {
9         timeout--;
10    }
11    DMAEndOfTransfer = 0x00;
12    timeout = SD_DATATIMEOUT;
13    while (((SDIO->STA & SDIO_FLAG_TXACT)) && (timeout > 0)) {
14        timeout--;
15    }
16    if (StopCondition == 1) {
17        errorstatus = SD_StopTransfer();
18        StopCondition = 0;
19    }
20    if ((timeout == 0) && (errorstatus == SD_OK)) {
21        errorstatus = SD_DATA_TIMEOUT;
```

```
22     }
23     /*!< Clear all the static flags */
24     SDIO_ClearFlag(SDIO_STATIC_FLAGS);
25     if (TransferError != SD_OK) {
26         return (TransferError);
27     } else {
28         return (errorstatus);
29     }
30 }
```

该函数开始等待当前块数据正确传输完成，并添加了超时检测功能。然后不停监测 SDIO_STA 寄存器的 TXACT 位，以等待数据写入完成。对于多块数据写入操作需要调用 SD_StopTransfer 函数停止数据传输，而单块写入则不需要。SD_StopTransfer 函数实际是向 SD 卡发送 CMD12，该命令专门用于停止数据传输，SD 卡系统保证在主机发送 CMD12 之后整块传输完后才停止数据传输。SD_WaitWriteOperation 函数最后是清除相关标志位并返回错误。

数据读取操作

同向 SD 卡写入数据类似，从 SD 卡读取数据可分为单块读取和多块读取。这里这介绍单块读操作函数，多块读操作类似理解即可。

代码清单 37-13 SD_ReadBlock 函数

```
1 SD_Error SD_ReadBlock(uint8_t *readbuff, uint64_t ReadAddr,
2                               uint16_t BlockSize)
3 {
4     SD_Error errorstatus = SD_OK;
5     TransferError = SD_OK;
6     TransferEnd = 0;
7     StopCondition = 0;
8
9     SDIO->DCTRL = 0x0;
10    #if defined (SD_DMA_MODE)
11        SDIO_ITConfig(SDIO_IT_DCRCFAIL | SDIO_IT_DTIMEOUT | SDIO_IT_DATAEND |
12                      SDIO_IT_RXOVERR | SDIO_IT_STBITERR, ENABLE);
13        SDIO_DMACmd(ENABLE);
14        SD_LowLevel_DMA_RxConfig((uint32_t *)readbuff, BlockSize);
15    #endif
16    if (CardType == SDIO_HIGH_CAPACITY_SD_CARD) {
17        BlockSize = 512;
18        ReadAddr /= 512;
19    }
20    /* Set Block Size for Card */
21    SDIO_CmdInitStructure.SDIO_Argument = (uint32_t) BlockSize;
22    SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_SET_BLOCKLEN;
23    SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
24    SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
25    SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
26    SDIO_SendCommand(&SDIO_CmdInitStructure);
27    errorstatus = CmdResp1Error(SD_CMD_SET_BLOCKLEN);
28    if (SD_OK != errorstatus) {
29        return (errorstatus);
30    }
31    SDIO_DataInitStructure.SDIO_DataTimeOut = SD_DATATIMEOUT;
32    SDIO_DataInitStructure.SDIO_DataLength = BlockSize;
33    SDIO_DataInitStructure.SDIO_DataBlockSize = (uint32_t) 9 << 4;
34    SDIO_DataInitStructure.SDIO_TransferDir = SDIO_TransferDir_ToSDIO;
35    SDIO_DataInitStructure.SDIO_TransferMode = SDIO_TransferMode_Block;
36    SDIO_DataInitStructure.SDIO_DPSM = SDIO_DPSM_Enable;
37    SDIO_DataConfig(&SDIO_DataInitStructure);
38    /*!< Send CMD17 READ_SINGLE_BLOCK */
39    SDIO_CmdInitStructure.SDIO_Argument = (uint32_t) ReadAddr;
40    SDIO_CmdInitStructure.SDIO_CmdIndex = SD_CMD_READ_SINGLE_BLOCK;
41    SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short;
```

```
42     SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
43     SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable;
44     SDIO_SendCommand(&SDIO_CmdInitStructure);
45     errorstatus = CmdResplError(SD_CMD_READ_SINGLE_BLOCK);
46     if (errorstatus != SD_OK) {
47         return (errorstatus);
48     }
49     return (errorstatus);
50 }
```

数据读取操作与数据写入操作编程流程是类似，只是数据传输方向改变，使用到的 SD 命令号也有所不同而已。SD_ReadBlock 函数有三个形参，分别为数据读取存储器的指针、数据读取起始目标地址和单块长度。SD_ReadBlock 函数执行流程如下：

- (1) 将 SDIO 外设的数据控制寄存器 (SDIO_DCTRL) 清理，复位之前的传输设置。
- (2) 调用 SDIO_ITConfig 函数使能相关中断，包括数据 CRC 失败中断、数据超时中断、数据结束中断等等。然后调用 SD_LowLevel_DMA_RxConfig 函数，配置使能 SDIO 从 SD 卡的读取数据的 DMA 请求，该函数可以参考代码清单 37-6。为使 SDIO 发送 DMA 请求，需要调用 SDIO_DMACmd 函数使能。对于高容量的 SD 卡要求块大小必须为 512 字节，程序员有责任保证目标读取地址与块大小的字节对齐问题。
- (3) 对 SD 卡进行数据读写之前，都必须发送 CMD16 指定块的大小，对于标准卡，读取 BlockSize 长度字节的块；对于 SDHC 卡，读取 512 字节的块。
- (4) 利用 SDIO_DataInitTypeDef 结构体类型变量配置数据传输的超时、块数量、数据块大小、数据传输方向等参数并使用 SDIO_DataConfig 函数完成数据传输环境配置。
- (5) 最后控制器向 SD 卡发送单块读数据命令 CMD17，SD 卡在接收到命令后就会通过数据线把数据传输到控制器数据 FIFO 内，并自动生成 DMA 传输请求。

读取操作等待函数

SD_WaitReadOperation 函数用于等待数据读取操作完成，只有在确保数据读取完成了我们才可以放心使用数据。SD_WaitReadOperation 函数也是适用于单块读取函数和多块读取函数的。

代码清单 37-14 SD_WaitReadOperation 函数

```
1 SD_Error SD_WaitReadOperation(void)
2 {
3     SD_Error errorstatus = SD_OK;
4     uint32_t timeout;
5
6     timeout = SD_DATATIMEOUT;
7     while ((DMAEndOfTransfer == 0x00) && (TransferEnd == 0) &&
8           (TransferError == SD_OK) && (timeout > 0)) {
9         timeout--;
10    }
11
12    DMAEndOfTransfer = 0x00;
13    timeout = SD_DATATIMEOUT;
14    while (((SDIO->STA & SDIO_FLAG_RXACT)) && (timeout > 0)) {
15        timeout--;
16    }
17    if (StopCondition == 1) {
18        errorstatus = SD_StopTransfer();
19        StopCondition = 0;
20    }
21    if ((timeout == 0) && (errorstatus == SD_OK)) {
22        errorstatus = SD_DATA_TIMEOUT;
23    }
```

```
24      /*!< Clear all the static flags */
25      SDIO_ClearFlag(SDIO_STATIC_FLAGS);
26      if (TransferError != SD_OK) {
27          return (TransferError);
28      } else {
29          return (errorstatus);
30      }
31 }
```

该函数开始等待当前块数据正确传输完成，并添加了超时检测功能。然后不停监测 SDIO_STA 寄存器的 RXACT 位，以等待数据读取完成。对于多块数据读取操作需要调用 SD_StopTransfer 函数停止数据传输，而单块写入则不需要。该函数最后是清除相关标志位并返回错误。

5. SDIO 中断服务函数

在进行数据传输操作时都会使能相关标志中断，用于跟踪传输进程和错误检测。如果是使用 DMA 传输，也会使能 DMA 相关中断。为简化代码，加之 SDIO 中断服务函数内容一般不会修改，将中断服务函数放在 bsp_sdio_sd.c 文件中，而不是放在常用于存放中断服务函数的 stm32f4xx_it.c 文件。

代码清单 37-15 SDIO 中断服务函数

```
1 void SDIO_IRQHandler(void)
2 {
3     /* Process All SDIO Interrupt Sources */
4     SD_ProcessIRQSrc();
5 }
6
7 SD_Error SD_ProcessIRQSrc(void)
8 {
9     if (SDIO_GetITStatus(SDIO_IT_DATAEND) != RESET) {
10         TransferError = SD_OK;
11         SDIO_ClearITPendingBit(SDIO_IT_DATAEND);
12         TransferEnd = 1;
13     } else if (SDIO_GetITStatus(SDIO_IT_DCRCFAIL) != RESET) {
14         SDIO_ClearITPendingBit(SDIO_IT_DCRCFAIL);
15         TransferError = SD_DATA_CRC_FAIL;
16     } else if (SDIO_GetITStatus(SDIO_IT_DTIMEOUT) != RESET) {
17         SDIO_ClearITPendingBit(SDIO_IT_DTIMEOUT);
18         TransferError = SD_DATA_TIMEOUT;
19     } else if (SDIO_GetITStatus(SDIO_IT_RXOVERR) != RESET) {
20         SDIO_ClearITPendingBit(SDIO_IT_RXOVERR);
21         TransferError = SD_RX_OVERRUN;
22     } else if (SDIO_GetITStatus(SDIO_IT_TXUNDERR) != RESET) {
23         SDIO_ClearITPendingBit(SDIO_IT_TXUNDERR);
24         TransferError = SD_TX_UNDERRUN;
25     } else if (SDIO_GetITStatus(SDIO_IT_STBITERR) != RESET) {
26         SDIO_ClearITPendingBit(SDIO_IT_STBITERR);
27         TransferError = SD_START_BIT_ERR;
28     }
29
30     SDIO_ITConfig(SDIO_IT_DCRCFAIL | SDIO_IT_DTIMEOUT | SDIO_IT_DATAEND |
31                  SDIO_IT_TXFIFOHE | SDIO_IT_RXFIFOHF | SDIO_IT_TXUNDERR
32                  | SDIO_IT_RXOVERR | SDIO_IT_STBITERR, DISABLE);
33     return (TransferError);
34 }
```

SDIO 中断服务函数 SDIO_IRQHandler 会直接调用 SD_ProcessIRQSrc 函数执行。SD_ProcessIRQSrc 函数通过多个 if 判断语句分辨中断源，并对传输错误标志变量 TransferError 赋值以指示当前传输状态。最后禁用 SDIO 中断。

代码清单 37-16 DMA 请求中断

```
1 void SD_SDIO_DMA_IRQHANDLER(void)
2 {
3     /* Process DMA2 Stream3 or DMA2 Stream6 Interrupt Sources */
4     SD_ProcessDMAIRQ();
5 }
6
7 void SD_ProcessDMAIRQ(void)
8 {
9     if (DMA2->>LISR & SD_SDIO_DMA_FLAG_TCIF) {
10         DMAEndOfTransfer = 0x01;
11         DMA_ClearFlag(SD_SDIO_DMA_STREAM,
12                       SD_SDIO_DMA_FLAG_TCIF|SD_SDIO_DMA_FLAG_FEIF);
13     }
14 }
```

SD_SDIO_DMA_IRQHANDLER 函数是 DMA 传输中断服务函数，它直接调用 SD_ProcessDMAIRQ 函数执行。SD_ProcessDMAIRQ 函数主要是判断 DMA 的传输完成标志位。

至此，我们已经介绍了 SD 卡初始化、SD 卡数据操作的基础功能函数以及 SDIO 相关中断服务函数内容，很多时候这些函数已经足够我们使用了。接下来我们就编写一些简单的测试程序验证移植的正确性。

6. 测试函数

测试 SD 卡部分的函数是我们自己编写的，存放在 sdio_test.c 文件中。

SD 卡测试函数

代码清单 37-17 SD_Test

```
1 void SD_Test(void)
2 {
3     LED_BLUE;
4     /*----- SD Init ----- */
5     /* SD 卡使用 SDIO 中断及 DMA 中断接收数据，中断服务程序位于 bsp_sdio_sd.c 文件尾*/
6     if ((Status = SD_Init()) != SD_OK) {
7         LED_RED;
8         printf("SD 卡初始化失败，请确保 SD 卡已正确接入开发板，或换一张 SD 卡测试! \n");
9     } else {
10         printf("SD 卡初始化成功! \n");
11     }
12     if (Status == SD_OK) {
13         LED_BLUE;
14         /*擦除测试*/
15         SD_EraseTest();
16
17         LED_BLUE;
18         /*single block 读写测试*/
19         SD_SingleBlockTest();
20
21         /*暂不支持直接多块读写，多块读写可用多个单块读写流程代替*/
22         LED_BLUE;
23         /*muti block 读写测试*/
24         SD_MultiBlockTest();
25     }
26 }
```

测试程序以开发板上 LED 灯指示测试结果，同时打印相关测试结果到串口调试助手。测试程序先调用 SD_Init 函数完成 SD 卡初始化，该函数具体代码参考代码清单 37-7，如果初始化成功就可以进行数据操作测试。

SD 卡擦除测试

代码清单 37-18 SD_EraseTest

```
1 void SD_EraseTest(void)
2 {
3     /*----- Block Erase -----*/
4     if (Status == SD_OK) {
5         /* Erase NumberOfBlocks Blocks of WRITE_BL_LEN(512 Bytes) */
6         Status = SD_Erase(0x00, (BLOCK_SIZE * NUMBER_OF_BLOCKS));
7     }
8
9     if (Status == SD_OK) {
10        Status = SD_ReadMultiBlocks(Buffer_MultiBlock_Rx, 0x00,
11                                   BLOCK_SIZE, NUMBER_OF_BLOCKS);
12
13        /* Check if the Transfer is finished */
14        Status = SD_WaitReadOperation();
15
16        /* Wait until end of DMA transfer */
17        while (SD_GetStatus() != SD_TRANSFER_OK);
18    }
19
20    /* Check the correctness of erased blocks */
21    if (Status == SD_OK) {
22        EraseStatus = eBuffercmp(Buffer_MultiBlock_Rx, MULTI_BUFFER_SIZE);
23    }
24
25    if (EraseStatus == PASSED) {
26        LED_GREEN;
27        printf("SD 卡擦除测试成功! \n");
28    } else {
29        LED_BLUE;
30        printf("SD 卡擦除测试失败! \n");
31        printf("温馨提示: 部分 SD 卡不支持擦除测试, 若 SD 卡能通过下面的 single \
32                读写测试, 即表示 SD 卡能够正常使用.\n");
33    }
34 }
```

SD_EraseTest 函数主要编程思路是擦除一定数量的数据块, 接着读取已擦除块的数据, 把读取到的数据与 0xff 或者 0x00 比较, 得出擦除结果。

SD_Erase 函数用于擦除指定地址空间, 源代码参考代码清单 37-10, 它接收两个参数指定擦除空间的起始地址和终止地址。如果 SD_Erase 函数返回正确, 表示擦除成功则执行数据块读取; 如果 SD_Erase 函数返回错误, 表示 SD 卡擦除失败, 并不是所有卡都能擦除成功的, 部分卡虽然擦除失败, 但数据读写操作也是可以正常执行的。这里使用多块读取函数 SD_ReadMultiBlocks, 它有四个形参, 分别为读取数据存储器、读取数据目标地址、块大小以及块数量, 函数后面都会跟随等待数据传输完成相关处理代码。接下来会调用 eBuffercmp 函数判断擦除结果, 它有两个形参, 分别为数据指针和数据字节长度, 它实际上是把数据存储器内所有数据都与 0xff 或 0x00 做比较, 只有出现这两个数之外就报错退出。

单块读写测试

代码清单 37-19 SD_SingleBlockTest 函数

```
1 void SD_SingleBlockTest(void)
2 {
3     /*----- Block Read/Write -----*/
4     /* Fill the buffer to send */
5     Fill_Buffer(Buffer_Block_Tx, BLOCK_SIZE, 0x320F);
6 }
```

```
7   if (Status == SD_OK) {
8       /* Write block of 512 bytes on address 0 */
9       Status = SD_WriteBlock(Buffer_Block_Tx, 0x00, BLOCK_SIZE);
10      /* Check if the Transfer is finished */
11      Status = SD_WaitWriteOperation();
12      while (SD_GetStatus() != SD_TRANSFER_OK);
13  }
14  if (Status == SD_OK) {
15      /* Read block of 512 bytes from address 0 */
16      Status = SD_ReadBlock(Buffer_Block_Rx, 0x00, BLOCK_SIZE);
17      /* Check if the Transfer is finished */
18      Status = SD_WaitReadOperation();
19      while (SD_GetStatus() != SD_TRANSFER_OK);
20  }
21  /* Check the correctness of written data */
22  if (Status == SD_OK) {
23      TransferStatus1 = Buffercmp(Buffer_Block_Tx,
24                                  Buffer_Block_Rx, BLOCK_SIZE);
25  }
26  if (TransferStatus1 == PASSED) {
27      LED_GREEN;
28      printf("Single block 测试成功! \n");
29  } else {
30      LED_RED;
31      printf("Single block 测试失败, 请确保 SD 卡正确接入开发板, 或换一张 SD 卡测试! \n");
32  }
33 }
```

SD_SingleBlockTest 函数主要编程思想是首先填充一个块大小的存储器, 通过写入操作把数据写入到 SD 卡内, 然后通过读取操作读取数据到另外的存储器, 然后在对比存储器内容得出读写操作是否正确。

SD_SingleBlockTest 函数一开始调用 Fill_Buffer 函数用于填充存储器内容, 它只是简单实用 for 循环赋值方法给存储区填充数据, 它有三个形参, 分别为存储区指针、填充字节数和起始数选择, 这里的起始数选择参数对本测试没有实际意义。SD_WriteBlock 函数和 SD_ReadBlock 函数分别执行数据写入和读取操作, 具体可以参考代码清单 37-11 和代码清单 37-13。Buffercmp 函数用于比较两个存储区内容是否完全相等, 它有三个形参, 分别为第一个存储区指针、第二个存储区指针和存储器长度, 该函数只是循环比较两个存储区对应位置的两个数据是否相等, 只有发现存在不相等就报错退出。

SD_MultiBlockTest 函数与 SD_SingleBlockTest 函数执行过程类似, 这里就不做详细分析。

主函数

代码清单 37-20 main 函数

```
1 int main(void)
2 {
3
4     /* 初始化 LED 灯 */
5     LED_GPIO_Config();
6     LED_BLUE;
7     /* 初始化独立按键 */
8     Key_GPIO_Config();
9
10    /*初始化 USART1*/
11    Debug_USART_Config();
12
13    printf("\r\n 欢迎使用野火  STM32  F407  开发板.\r\n");
14
15    printf("在开始进行 SD 卡基本测试前, 请给开发板插入 32G 以内的 SD 卡\r\n");
```

```
18     printf("本程序会对 SD 卡进行 非文件系统 方式读写, 会删除 SD 卡的文件系统\r\n");
19     printf("实验后可通过电脑格式化或使用 SD 卡文件系统的例程恢复 SD 卡文件系统\r\n");
20     printf("\r\n 但 sd 卡内的原文件不可恢复, 实验前务必备份 SD 卡内的原文件!!! \r\n");
21
22     printf("\r\n 若已确认, 请按开发板的 KEY1 按键, 开始 SD 卡测试实验....\r\n");
23
24     /* Infinite loop */
25     while (1) {
26         /*按下按键开始进行 SD 卡读写实验, 会损坏 SD 卡原文件*/
27         if ( Key_Scan(KEY1_GPIO_PORT,KEY1_PIN) == KEY_ON) {
28             printf("\r\n 开始进行 SD 卡读写实验\r\n");
29             SD_Test();
30         }
31     }
32 }
```

测试过程中有用到 LED 灯、独立按键和调试串口, 所以需要对这些模块进行初始化配置。在无限循环中不断检测按键状态, 如果有被按下就执行 SD 卡测试函数。

37.9.3 下载验证

把 Micro SD 卡插入到开发板右侧的卡槽内, 使用 USB 线连接开发板上的“USB TO UART”接口到电脑, 电脑端配置好串口调试助手参数。编译实验程序并下载到开发板上, 程序运行后在串口调试助手可接收到开发板发过来的提示信息, 按下开发板左下边沿的 K1 按键, 开始执行 SD 卡测试, 测试结果在串口调试助手可观察到, 板子上 LED 灯也可以指示测试结果。